

VILNIUS UNIVERSITY
KAUNAS FACULTY

INSTITUTE OF SOCIAL SCIENCES AND APPLIED INFORMATICS

Study programme Information Systems and Cyber Security
State code 6121BX003

NAYAN JOSHY MANIYATH JOSHY

BACHELOR'S THESIS

A SYSTEM FOR DETECTING CRYPTOCURRENCY TOKEN SCAMS

Kaunas 2025

VILNIUS UNIVERSITY
KAUNAS FACULTY

INSTITUTE OF SOCIAL SCIENCES AND APPLIED INFORMATICS

NAYAN JOSHY MANIYATH JOSHY

BACHELOR'S THESIS

A SYSTEM FOR DETECTING CRYPTOCURRENCY TOKEN SCAMS

Allowed to defend	_____	Bachelor student	<u>Nayan Joshy Maniyath Joshy</u> (signature)
		Scientific advisor	<u>Paulius Danielius</u> (signature)
			<u>(scientific degree of the advisor, scientific pedagogical name, name and surname)</u>
		Thesis submitted on	<u>22/12/2025</u>
		Registration No:	_____

Kaunas 2025

TABLE OF CONTENTS

LIST OF FIGURES	6
LIST OF TABLES	7
LIST OF ABBREVIATIONS	8
SANTRAUKA	9
INTRODUCTION	11
1. ANALYTICAL PART	13
1.1 Analysis of the General Object Characteristics.....	13
1.2 Tokenomics concept and risk factors	14
1.2.1 Understanding common types of crypto scams.....	14
1.2.2 Case studies of major meme coin incidents	16
1.3 Analysis of functions, Information flows and existing solutions.....	17
1.3.1 Shortcomings and Discrepancies of Existing Solutions.....	18
1.3.2 Market analysis of existing solutions	19
1.4 Business Model and Monetization Strategy	22
1.4.1 Tiered Access Structure	22
1.4.2 Operational Cost Structure	22
1.5 Justification of Selected Development Tools	24
2. Technical Task	26
3. PROJECT PART	28
3.1 Purpose of the project.....	28
3.2 Logical Structure of the System	28
3.3 Functional Requirements.....	30
3.4 Non-Functional Requirements	31
3.5 Use case diagram of the Tokenomics Lab Platform.....	33

3.6	Computerised system workflow	35
3.7	Hierarchy Of Computerized Functions	36
3.8	Data flow diagram of computerised system	37
3.8.1	Context Diagram (Level 0 DFD)	38
3.8.2	Token Analysis Data Flow (Level 1).....	39
3.8.3	Payment and RBAC Data Flow (Level 1 DFD).....	40
3.9	Conceptual object model	41
3.10	System states and processes	43
3.10.1	Core system states	43
3.10.2	Processes and transitions	44
3.10.3	Formal description of calculations	44
3.10.4	10-Factor Risk calculation Breakdown	45
3.10.5	Chain-Adaptive Weights and Overrides	46
3.10.6	Post-Calculation Overrides.....	48
3.10.7	Final Score Calculation and Risk Bands	48
4.	Part of software implementation	50
4.1	Implementation environment	50
4.2	Database Specification	50
4.2.1	Users Collection Schema	51
4.2.2	Watchlist Sub-Collection Schema	52
4.2.3	Analysis History Sub-Collection Schema	52
4.2.4	Activity Logs Sub-Collection Schema	53
4.2.5	Credit System Collections	53
4.2.6	Alert Notifications Sub-Collection.....	54
4.2.7	Entity Relationship Diagram of Core Firestore Collections	55
4.3	Core Firestore Security Rules Configuration	55
4.4	User Interface Modules and Relationships.....	58
4.5	User Dashboards and Result View Interfaces	59

4.6	Input data specification	68
4.7	Output data specification.....	70
4.8	Information processing.....	71
4.9	Data Architecture and GDPR Compliance Strategy	72
4.9.1	Data Collection and Minimization Policy	72
4.9.2	Technical Implementation of User Rights.....	73
4.9.3	Data Security and Processor Management.....	73
4.10	Automated Token Classification and Narrative Detection.....	73
4.11	Testing and Validation	74
4.11.1	Risk Algorithm Simulation.....	74
4.11.2	Beta User Testing.....	75
4.11.3	Manual functional testing of the payment system	76
4.11.4	UI and Functionality Testing	78
4.12	Users guide.....	78
4.12.1	Getting Started & Registration	79
4.12.2	Analyzing a Token.....	80
4.12.3	Understanding the Analysis Report.....	82
4.12.4	Portfolio Audit (Premium Feature).....	83
4.12.5	Plans, Payments & Upgrading.....	84
4.12.6	User Profile Management.....	85
4.12.7	Admin Panel (Restricted)	87
4.13	Programmer's Guide	89
5.	Conclusions	91
	References	92
	SUMMARY.....	94
	ANNEXE.....	95

LIST OF FIGURES

Figure 1: Cryptocurrency-related losses reported to IC3 in 2024	15
Figure 2:High-level Data Flow Diagram of Tokenomics Lab Platform.....	18
Figure 3:Use case diagram of Tokenomics lab.....	33
Figure 4:Activity Diagram of the system	35
Figure 5:Platform Hierarchy Diagram.....	36
Figure 6:Context diagram.....	38
Figure 7:Token Analysis Data Flow (Level 1)	39
Figure 8:Payment and RBAC Data Flow diagram (Level)	40
Figure 9:ERD diagram	41
Figure 10: Entity Relationship Diagram of Core Firestore Collections	55
Figure 11:Higherachy of the ui Componenets.....	58
Figure 12:Landing page.....	60
Figure 13:Navigation menu user inetrface	61
Figure 14:Free user's dashboard	62
Figure 15:Premium Users Dashboard	63
Figure 16:Pay-As-You-Go users Dashboard	64
Figure 17:Token result view	65
Figure 18:Token Explorer UI	66
Figure 19:Admin pannel of the platform.....	67
Figure 20:Algorithm Vailidation result	75
Figure 21:Phantom wallet transaction history	77
Figure 22:Admin panel recent transaction view.....	78
Figure 23:Sign up page.....	79
Figure 24:Token Explorer.....	81
Figure 25:Risk interpretation.....	82
Figure 26:portfolio audit pannel.....	83
Figure 27:Pricing page	84
Figure 28:Payment page	84
Figure 29>User profile management.....	85
Figure 30:Admin pannel view	87

LIST OF TABLES

Table 1 Comparative analysis of token risk and analytics platforms	21
Table 2 API Pricing & Utilization Table	23
Table 3 Logical structure of the system layers and components	29
Table 4 Functional requirements of the proposed system	30
Table 5 Non-Functional requirements of the proposed system	31
Table 6 Table of Risk factors and calculations	45
Table 7 Chain-Adaptive Weight Distribution by Ecosystem.....	47
Table 8 Table of Risk Bands.....	49
Table 9 Table of Technology Stack.....	50
Table 10 Table of Users Collection	51
Table 11 Table of Watchlist Sub-Collection Schema.....	52
Table 12 Table of Analysis History Sub-Collection Schema	52
Table 13 Table of Activity Logs Sub-Collection Schema	53
Table 14 Table of Credit System Collection.....	53
Table 15 Table of Credit System Collections Transactions.....	53
Table 16 Table of Alert Notifications Sub-Collections.....	54

LIST OF ABBREVIATIONS

- AI** – Artificial Intelligence (Dirbtinis intelektas)
- API** – Application Programming Interface (Programų sąsaja)
- DeFi** – Decentralized Finance (Decentralizuoti finansai)
- EVM** – Ethereum Virtual Machine (Ethereum virtualioji mašina)
- GDPR** – General Data Protection Regulation (Bendrasis duomenų apsaugos reglamentas)
- HTTP** – Hypertext Transfer Protocol (Hiperteksto perdavimo protokolas)
- JSON** – JavaScript Object Notation
- LLM** – Large Language Model (Didysis kalbos modelis)
- RPC** – Remote Procedure Call (Nuotolinis procedūros kvietimas)
- SDK** – Software Development Kit (Programinės įrangos kūrimo rinkinys)
- UI** – User Interface (Naudotojo sąsaja)
- UX** – User Experience (Naudotojo patirtis)
- USD** – United States Dollar (JAV doleris)

Maniyath Joshy, Nayan Joshy. (2025) A System For Detecting Cryptocurrency Token Scams. Bachelor's Thesis. Kaunas: Vilnius University Kaunas Faculty. [101] pages,[16] tables, and [30] figures..

SANTRAUKA

Šio darbo tikslas suprojektuoti ir įgyvendinti „Tokenomics Lab“, holistinę internetinę informacinę sistemą, skirtą automatizuoti kriptovaliutų žetonų rizikos analizę keliuose blokų grandinių tinkluose. Tyrimu sprendžiama decentralizuotų finansų (DeFi) sektoriaus fragmentacijos problema, kai mažmeniniai investuotojai neturi vieningų įrankių sukčiavimams, tokiems kaip „rug pulls“ ir „honeypots“, aptikti.

Šiam tikslui pasiekti buvo iškelti ir įgyvendinti šie uždaviniai: ištirti bendrus tokenomikos rizikos veiksnius, sukurti individualų svertinį 10 veiksnių rizikos vertinimo algoritmą, sukurti programą naudojant bevariklinę (*serverless*) architektūrą ir patvirtinti sistemos tikslumą.

Tyrimo metodologija rėmėsi „Agile“ / „Scrum“ kūrimo sistema. Sistema buvo sukurta naudojant modernų technologijų rinkinį, įskaitant „Next.js 16“ vartotojo sąsajai (*frontend*), „Firebase“ duomenų bazei ir autentifikavimui (*backend*) bei „Groq AI SDK“ („Llama 3.3“) natūralios kalbos apdorojimui. Buvo įdiegta „grandinei adaptyvi“ (*Chain-Adaptive*) architektūra, skirta normalizuoti duomenis iš skirtingų šaltinių („Ethereum“, „Solana“ ir kt.). Sistemos veikimas buvo įvertintas atliekant algoritminius modeliavimus su žinomų kenkėjiškų ir saugių žetonų duomenų rinkiniu, taip pat atliekant kokybinį beta testavimą su vartotojais.

Pagrindinis rezultatas – visiškai funkcionali platforma, kuri apjungia realaus laiko rinkos ir saugumo duomenis, kad sugeneruotų išsamų rizikos profilį. Sistema sėkmingai apskaičiuoja rizikos balą skalėje nuo 0 iki 100 ir pateikia dirbtinio intelekto sugeneruotus naratyvinius paaiškinimus. Testavimas parodė, kad rizikos algoritmas pasiekė 80 % tikslumą identifikuojant sukčiavimo žetonus, kas patvirtina algoritminio vertinimo derinimo su didžiaisiais kalbos modeliais (LLM) efektyvumą.

Išvados: „Tokenomics Lab“ platforma sėkmingai užpildo informacijos spragą DeFi rinkoje. Ji įrodo, kad sudėtingi grandinės duomenys gali būti apdoroti ir supaprastinti mažmeniniams investuotojams neprarandant analitinio gylio, taip suteikiant perspektyvų sprendimą automatizuotam sukčiavimo aptikimui.

Darbo apimtis: Darbą sudaro [104] puslapių, [16] lentelių ir [30] paveikslai.

Raktiniai žodžiai: Kripto valiuta, Tokenomika, Sukčiavimo aptikimas, DeFi, Dirbtinis intelektas, Rizikos analizė.

INTRODUCTION

Over the last decade, the cryptocurrency market has evolved from a niche technical sector to an established financial sector within the global financial landscape. This sudden growth in the adoption of defi has resulted in a proportional increase in various types of crypto frauds, ranging from simple rug pulls to advanced pump and dump schemes. It has never been easier for cyber criminals to create fraudulent cryptocurrency tokens to drain investor funds without the need for any sophisticated technical knowledge of blockchain technology. Anyone with an internet-enabled device can launch a fraudulent token market it as a legitimate and trick the retail investors into buying it and successfully drain their funds via their elaborate schemes.

Existing tools overwhelm the users with raw data and other unnecessary metrics which, rather than help the user, confuse them due to the lack of an easy-to-digest explanation and complex financial charts. This creates a significant problem for early-stage investors and crypto enthusiasts to effectively analyse the risk involved in their investments. Another issue is the lack of data interoperability. In the crypto space, different crypto tokens exist on different native blockchains, and each of these blockchains has its own ecosystem, which makes it harder to collect and analyse tokenomic data linked to these tokens.

The objective of this thesis is to design and implement Tokenomics Lab, a multi-chain information system dedicated to cryptocurrency token risk analysis.

The thesis aims to create a platform that automatically gathers and analyses financial data from multiple sources, generating understandable risk assessments for retail investors and crypto enthusiasts.

To achieve this aim, the following objectives have been set:

1. To investigate the common tokenomic factors behind crypto scams and identify where the current risk assessment tools in the market fall short.
2. To define the specific requirements for a system that can handle data from multiple blockchains simultaneously without sacrificing accuracy.
3. To design a custom weighted risk scoring algorithm that mainly utilises tokenomic factors to predict risk with significant accuracy.
4. To build the application using Next.js, integrating the risk calculation algorithm alongside a specialised AI model for natural language explanations and meme detection.
5. To test the system's accuracy against real-world historical data and validate its usability with a group of beta testers.

The research methodology included analysing available theoretical data, reviewing current literature on the topic, and directly comparing the results with market-leading tools. The development of the website followed a cycle of continuous development and rigorous testing. Finding appropriate sources for reliable multichain tokenomic data was the most challenging aspect of the whole process. This led to multiple revisions and integration of multiple API endpoints for the most optimal result.

The main result is a working web application, built on a modern tech stack focused on speed and security, that identifies high-risk tokens with a demonstrated 80% accuracy during testing. This demonstrates how Large Language Models can be effectively combined with traditional algorithmic scoring.

1. ANALYTICAL PART

The following section analyzes the theoretical and practical aspects of the research object, covering tokenomics risk factors, market alternatives, and the technical specifications required to develop the fraud detection platform

1.1 Analysis of the General Object Characteristics

In this thesis, the research object is the Tokenomics Lab platform, a web-based information system designed to analyse the risk of cryptocurrency tokens across several blockchain networks. The system operates as a Next.js application, where authenticated users can submit a token address or query the token by name and receive a structured risk assessment of its risk profile. From the user's perspective, the website acts as an interactive dashboard with easy-to-understand explanations and in-depth risk calculations while being user-friendly and accurate.

The platform is primarily targeted towards meme coins and other high-risk tokens, where traditional metrics are weak but tokenomics and market structure still provide meaningful risk signals. Tokenomics Lab targets individual investors, analysts and advanced retail traders who need more than price charts to evaluate whether a token is worth the risk. The platform supports multiple major networks, including Ethereum, BNB Smart Chain, Solana, Polygon and Cardano, and applies the same 0–100 risk scale across all of them. Internally, the system aggregates market data, holder statistics, liquidity information and contract-level security indicators from various APIs such as Mobula, Moralis, GoPlus, Helius and CoinGecko, normalises this information into a common token data model, and passes it to a ten-factor risk calculation engine. This engine combines factors like supply dilution, holder concentration, liquidity depth and adoption into a weighted score, and then enriches the result with AI-generated explanations using the Groq Llama 3.3 model. (Ghattas, 2025)

From an developers perspective, the platform is organised into a client layer (Next.js/React), an API route layer, a business logic layer and a data layer based on Firebase Authentication and Firestore. Role-based access control distinguishes between free, premium and administrator users, enabling tiered functionality such as watchlists, alerts and administrative analytics. The system must therefore satisfy not only functional requirements related to token risk analysis, but also non-functional requirements on performance, reliability, security and GDPR-compliant handling of user data, as specified in the university's methodological guidelines for information systems theses.

1.2 Tokenomics concept and risk factors

Tokenomics describes the economic design of a cryptocurrency token, including its supply model, distribution, incentives for holders and traders, and mechanisms intended to influence price and behaviour over time. Unlike traditional equities, many tokens are created with highly flexible supply rules, discretionary team allocations and aggressive marketing incentives, which can create significant hidden risks for retail investors. For this reason, an analysis limited to price charts or basic contract flags is insufficient; it is necessary to evaluate the underlying economic structure of a token as well.

The Tokenomics Lab platform adopts a ten-factor model that operationalises tokenomics into measurable risk indicators. Supply-related factors such as supply dilution and vesting unlocks describe how much of the total supply is already in circulation and how much may enter the market later, potentially creating sell pressure. Holder concentration and distribution patterns capture the extent to which a small number of “whales” can control price movements or perform coordinated exits. Liquidity depth reflects how easily a position can be entered or exited without causing extreme slippage, while adoption metrics approximate whether the token is used or merely traded speculatively.

Additional factors such as tax and fee mechanisms, burn or deflation policies, audit transparency and contract control address more specific tokenomics decisions that often appear in meme tokens and experimental DeFi projects. High transaction taxes, for example, can discourage normal trading and lock users into illiquid positions, whereas frequent burns may create an illusion of scarcity without addressing concentration or liquidity issues. By structuring these aspects into a consistent factor set with defined weights, Tokenomics Lab translates abstract tokenomics concepts into a quantitative risk score that can be compared across multiple blockchains

1.2.1 Understanding common types of crypto scams

As per the (Federal bureau of investigation, 2024) estimates about 9.3 billion USD in losses linked to cryptocurrency-related investment fraud in 2024, while total reported losses from all internet crime reached 16.6 billion USD. Other crypto-crime overviews for 2024–2025 (Chainalysis Team, 2025) also state that several billion dollars per year continue to flow into on-chain fraud schemes, especially in DeFi and low-cap tokens on cheaper blockchains. Many of these scams share similar

structural features: a few wallets hold most of the supply, liquidity is thin, and insiders have much better exit options than normal users.

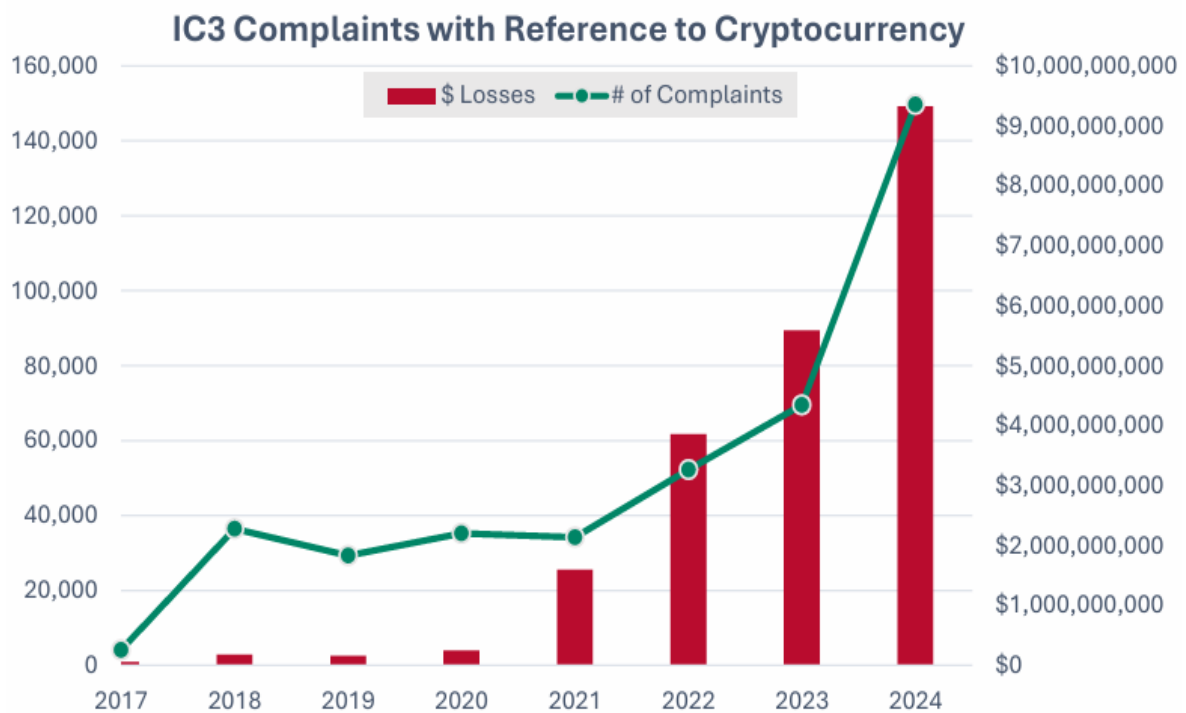


Figure 1: Cryptocurrency-related losses reported to IC3 in 2024

Source: Compiled by the author according to (Federal bureau of investigation, 2024)

The first group are rug pulls and exit scams. As described by (Finshots, 2025) In a typical rug pull, developers launch a token, add some initial liquidity, promote the project aggressively and then remove the liquidity or dump their own tokens once enough outside money has entered. After this, the price collapses and ordinary holders are left with tokens they cannot realistically sell. One recent summary based on Chainalysis data estimates that roughly 5 % of all tokens issued in 2024 were rug pulls, with investor losses of about 2.8 billion USD. A separate 2025 article (Merkle Science reports \$500M lost to memecoin scams in 2024, 2025), drawing on a report by Merkle Science, states that investors lost more than 500 million USD specifically to meme coin rug pulls and similar scams in 2024, many of them promoted through fake or hacked celebrity accounts. On-chain, these projects often show very high holder concentration and low liquidity compared to market cap and contracts, where the owner still controls critical functions.

The second group are pump-and-dump schemes on low-liquidity tokens. In these schemes, a small group accumulates a large share of the supply of their token, then artificially pumps the price through coordinated buying, hype and misleading marketing, and finally dumps their tokens on late

buyers when the price of the token reaches a certain amount. Research on decentralised exchanges shows that a noticeable share of new tokens exhibits pump-and-dump-like price and volume patterns, especially where liquidity is shallow, and one address later pulls most of it. These schemes depend on the same ingredients as many rugs pulls: a few large holders, weak liquidity and short-term speculative interest.

Another important group are Ponzi-style and high-yield tokens, where the tokenomics design itself is part of the fraud. These projects typically promise very high, sometimes guaranteed returns if users buy and lock a native token in staking, lending or similar contracts. Official 2024 statistics show that investment scams, including many high-yield crypto schemes, were the largest single fraud category, with reported losses of around 6.6 billion USD in that year. On-chain, such tokens often have large insider allocations, upcoming vesting unlocks and very uneven distribution, which allows insiders to exit early while ordinary holders remain locked in. (Federal bureau of investigation, 2024)

Finally, there is a wide class of illiquid or “dead” tokens that may not end with a dramatic rug pull but still trap investors. After an initial hype phase, trading volume and on-chain activity can drop to almost zero, while liquidity is removed or reduced to very low levels. Even if the contract is not openly malicious, exiting a position in such a token can be nearly impossible without extreme slippage.

1.2.2 Case studies of major meme coin incidents

To better understand the risks faced by retail traders in the meme coin segment, it is useful to analyse several high-profile cases from recent years. These examples show how concentrated ownership, unclear tokenomics and centralised liquidity can lead to large losses, even when there are strong public attention and marketing.

One widely discussed example is the \$TRUMP meme coin on Solana. Public reporting shows that the token was launched with a total supply of one billion units, but only around 200 million were initially made available to the public, while the remaining 800 million stayed under the control of companies linked to the Trump brand. This means that roughly 80 % of the supply remained in insider hands, allowing a small group to exert significant control over future price movements. Analyses of on-chain activity also indicate that a few early “whale” wallets accumulated large amounts during the initial stages, while many small buyers who entered later ended up with losses once the price stabilised or declined after the hype phase. In parallel, multiple unofficial Trump-themed tokens appeared on other chains, often reusing similar names or tickers, which increased confusion for ordinary users trying to distinguish between genuine and copycat projects. (Stan Kaminsky, 2025), (wikipedia, 2025)

Another important incident is the BALD meme coin on the Base network. BALD was launched in mid-2023 and experienced a very rapid price increase, with its liquidity pool reportedly reaching more than 20–25 million USD within a short period. However, shortly after this rise, the deployer removed most of the liquidity from the pool, causing the price to crash by over 90 % and leaving many traders with tokens they could no longer sell at a meaningful price. Investigations into the event highlight several structural issues: a single address provided and controlled most of the liquidity, there were no robust liquidity locks, and a large share of the token supply and trading volume was tied to a small group of addresses. These characteristics meant that the project could function as a typical rug pull, even though it initially appeared as a successful and popular meme coin. (Rob Behnke, 2023)

Both cases demonstrate that major meme coin incidents are often not caused by complex technical exploits, but by economic and structural decisions in token design and distribution. Extremely high insider allocations, centralised control over liquidity and rapid speculative inflows create conditions where retail investors can lose money very quickly once insiders decide to exit. In the following sections of the thesis, these observed patterns are used to justify the selection of specific risk factors and requirements for a system that analyses meme coin tokenomics and on-chain behaviour.

1.3 Analysis of functions, Information flows and existing solutions.

To understand why a new platform is necessary, it is first important to map out how information currently moves in the token analysis landscape and what specific functions are required to generate a reliable risk score. The operational flow of the Tokenomics Lab platform is fundamentally an exercise in orchestrating and standardizing these fragmented information flows. The required information flow can be broken down into three distinct stages:

1. **Input Data Flow:** The process begins with the capture of raw data. Currently, analysts must manually visit multiple sources to gather the necessary data. In the proposed system, this flow is automated but highly complex, as it requires gathering data from a diverse set of specialized APIs.
 - **Market Data:** Information regarding price, volume, market capitalization, and liquidity flows from market aggregators like the Mobula API.
 - **Contract Security Data:** For Ethereum and other EVM chains, critical security flags such as honeypot status, minting capabilities, and owner renouncement flow from specialized security providers like the GoPlus API.

- **Chain-Specific Data:** Unique architecture data, such as Mint authority or Freeze Authority status on Solana, must be retrieved from dedicated sources like the Helius API.
2. **Processing Flow:** Raw data is rarely useful on its own. The central information processing function required is Normalization. Since every external API returns data in a different format, the system must apply an Adapter Pattern to transform these inputs into a single, standardized CompleteTokenData object. Once normalized, the data flows into the Risk Calculation Engine, which weighs ten specific factors to produce a composite risk score.
 3. **Output data Flow:** The final stage is the delivery of the processed information. The system must convert the numerical score and the raw data points into a human-readable format. This involves sending the structured data to an AI Engine to generate a text-based summary, which is then presented to the user alongside the visual risk analytics dashboard.

The figure below illustrates a High-level data flow diagram. this was created with the assistance of (Google, 2025).

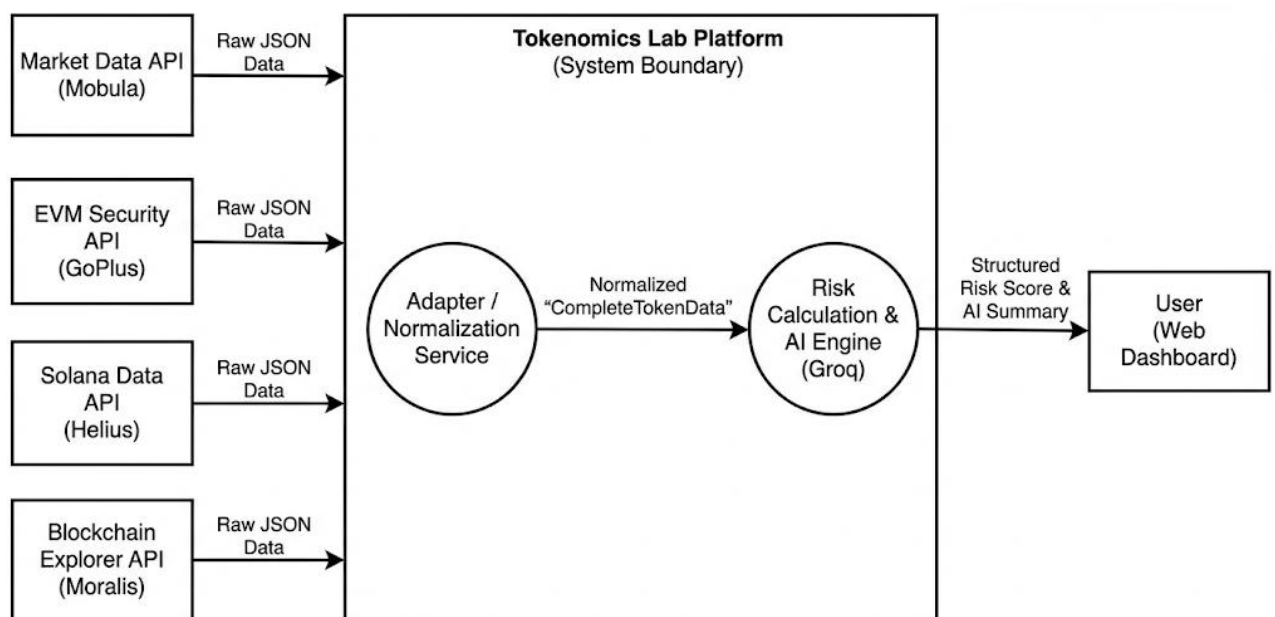


Fig. 2: High-level Information Flow Diagram of the Tokenomics Lab Platform.

Figure 2: High-level Data Flow Diagram of Tokenomics Lab Platform

Source: Created by the author with the help of Gemini 3 pro.

1.3.1 Shortcomings and Discrepancies of Existing Solutions

While the market tools analyzed in the previous section perform their individual functions well, a critical analysis reveals significant discrepancies between what these tools offer and what a regular investor needs to make a safe decision.

The main shortcomings identified are:

- **Fragmentation of Information:** There is a complete lack of functional unification. For example, GoPlus Security provides excellent contract analysis for Ethereum but cannot tell you if a Solana token has dangerous Freeze Authority. Conversely, Helius is excellent for Solana data but provides no insight into Ethereum honeypots. This fragmentation forces the user to act as the "system integrator," manually piecing together data from three or four different websites to get a complete picture.
- **Inconsistent Risk Standards:** Existing tools fail to normalize risk across different blockchains. A high-risk signal on an EVM chain like an unrenounced contract, is not directly comparable to a high-risk signal on Solana, like low liquidity within current platforms. There is no existing tool that applies a consistent 0–100 risk scale that adapts to the specific technical nuances of the underlying blockchain.
- **Lack of Context:** Perhaps the most significant gap is the lack of cognitive analysis. Current tools provide raw data telling the user "Tax is 20%" or "Liquidity is \$5,000", but they do not explain context. They often cannot classify a token and adjust the risk baseline accordingly. This leaves the user to interpret complex technical metrics on their own, often leading to incorrect assessment.
- **Inconsistent Data Quality:** Finally, the quality of data varies significantly between providers. Some APIs might return a null value for market cap, while others might have outdated holder counts. Existing solutions often simply display this raw, potentially erroneous data without validation. A robust system requires an internal logic layer to validate and "score" the quality of the data before presenting it to the user, a feature currently missing in the standard open-access tools.

These shortcomings confirm that simply having access to data is not enough. There is a clear need for an integrated information system that not only aggregates this multi-chain data but also normalizes, weighs, and explains it.

1.3.2 Market analysis of existing solutions

The current landscape of cryptocurrency analysis tools is highly segmented. While several platforms exist, they typically specialize in a single segment, either trading execution, static contract security, or institutional-grade wallet profiling, leaving a huge gap for retail investors seeking a simple and easy risk assessment.

The most common tools used by retail traders are decentralised exchange analytics platforms like DEXTools and DexScreener. These platforms specialize in providing real-time market data, including price data, volume metrics, and liquidity pool tracking across multiple chains. (Vermaak,

2022). They often provide basic trust scores based on liquidity and volume and can flag simple issues like a honeypot status. However, their risk analysis covers only the surface level of the issue. For instance, DexScreener focuses mainly on visualization and lacks a deep security audit engine, while DEXTools requires holding its native token to access detailed wallet information (Dextools, n.d.). More importantly, neither platform performs deep tokenomics verification, such as analyzing vesting schedules or insider distribution, which are critical for avoiding rug pulls.

For security-specific analysis, TokenSniffer and GoPlus Security serve as the industry standard for automated scanning. TokenSniffer is widely used to compare smart contracts against a database of known scams and check for permission flaws, such as hidden mint functions (Saher, 2024). Similarly, GoPlus operates as an API-first security layer, integrating into wallets to flag malicious addresses and contracts (Goplus, 2025). Despite their utility, these tools are often limited by blockchain support; TokenSniffer, for example, historically lacks full support for non-EVM chains like Solana (Token Sniffer, 2025). Furthermore, they typically provide raw "flags" without context. They do not correlate these flags with market data leaving the user to predict the risk level manually.

At the high end of the market are platforms like Nansen and CertiK. Nansen specializes in "Smart Money" labelling and wallet profiling, allowing users to track whale movements with high precision (Nansen, 2025). CertiK's Skynet provides comprehensive security ratings and has recently launched "SkyKnight" for Solana meme coin scanning (Certik, 2024). However, these tools face a significant barrier of complexity and cost. Nansen's deep analytics are designed for professional investors and VCs, presenting a steep learning curve and high subscription costs. CertiK's audit-grade scores are often delayed, making them less effective for the rapid lifecycle of a meme coin launch. Neither platform offers a simplified risk score specifically tailored for the immediate needs of a retail trader.

Here is a comparison table analyzing the features provided by major risk analysis platforms in the market today

Table 1

Comparative analysis of token risk and analytics platforms

Feature	TokenSniffer	CertiK (Skynet)	Nansen	Tokenomics Lab
Multi-Chain Scope	EVM only	Multi-chain (project-level monitoring)	Multi-chain (EVM + some non-EVM like Solana)	Multichain (EVM + Solana)
Risk Scoring Model	Static automated contract/liquidity score	Automated Skynet Score + audit	analytics & wallet signals	Chain-adaptive 10-factor risk score
AI Explanations	No	No	No per-token risk narratives	Yes (Groq-powered NL explanations)
Meme Token Detection	No dedicated classifier	No dedicated classifier	No dedicated classifier	Yes (AI based meme-token classification)
Portfolio Analysis	No	No	Yes (multi-chain portfolio tracker)	Yes (wallet-centric portfolio view)
Wallet Profiling	No	Limited to project/team info	Yes (labelled wallets, Smart Money, entities)	Yes (risk-focused wallet profiling)
Bulk Risk Scanning	Via API / scripts (not retail-oriented)	Enterprise monitoring across projects	Possible via analytics & API, not retail-focused	Yes (Via portfolio analysis)
Watchlist & Alerts	Basic (per-token checks, no rich alerts)	Yes (project-level security alerts, B2B)	Yes (Smart Alerts on paid tiers)	Integrated watchlists & risk alerts
Target Audience	Devs / degen traders	Institutions / projects / enterprises	Pro traders, funds, advanced users	Retail investors & active traders
Cost Accessibility	Free web scanner (paid API plans)	High-cost enterprise contracts	Freemium, paid subscriptions for full features	Freemium (advanced features on premium/pay-per-use model)

Source: compiled by the author.

1.4 Business Model and Monetization Strategy

The business model was shaped around a simple idea: most people want to try the platform with minimal friction, but a smaller group is willing to pay for depth and reliability. The result is a hybrid model that combines a generous free tier, on-demand credits, and a Pro subscription. This structure lets casual users verify a few tokens without commitment, while professional users use more advanced features.

1.4.1 Tiered Access Structure

The platform implements a three-tier access mechanism enforced through role-based authorization:

- **Free Tier:** This tier serves as the entry point, offering limited access to core features, with a daily limit of 20 scans. While this tier generates no direct revenue, it increases user retention and applying ethical rate limits to prevent abuse and automated scraping.
- **Pay-As-You-Go Tier:** Targeting occasional users, this tier utilizes x402 micropayment infrastructure on Solana blockchain. Users buy credits in small amount or more to unlock premium features such as AI-Generated Narratives and Portfolio Auditing. This model eliminates subscription friction while maintaining access to advanced tooling.
- **Pro Tier:** Designed for power users, the \$29/month subscription removes all usage restrictions and unlocks Advanced premium features such as Smart Alerts and Portfolio analysis and Monitoring. This tier provides the recurring revenue necessary to cover infrastructure overhead associated with real-time processing and webhook-driven notifications.

1.4.2 Operational Cost Structure

The platform's cost structure is divided into two components: variable costs tied to external data retrieval and fixed-plus-dynamic costs for infrastructure scaling.

External Data & API Costs (Variable): The system uses a Chain-Adaptive Fetcher to optimize API calls based on the specific blockchain context, ensuring that premium resources are only consumed when necessary.

The cost breakdown for external services is as follows:

- **Mobula (Market Data):** Covered under the free tier allowance; The cost negligible (Mobula, 2025).
- **GoPlus (Security Checks):** Open API with free access for standard throughput; \$0.00 per request (Goplus, 2025).
- **Helius (Solana RPC):** Subscription-based access; overhead absorbed into fixed costs. (Helius, 2025).

- **Groq AI (Risk Narrative):** Token-based pricing at approximately **\$0.08 per generated report** for a 200-token summary (groq, 2025).

Infrastructure Scalability (Fixed + Dynamic)

The application uses a serverless architecture to eliminate idle costs and scale dynamically with demand.

- **Frontend & Serverless Functions (Netlify):** The Starter plan (free) provides 125,000 serverless function per month. For higher traffic, the Pro plan (\$19/month) scales to 2, supporting thousands of users without overage charges (Netlify, 2025).
- **Database (Firebase Firestore):** Firestore operates on a usage-based model under the Blaze plan with read operations costing \$0.06 per 100,000 and write operations costing \$0.18 per 100,000 operations.
- For a typical user performing 20 scans daily, the database load consists of approximately 1 write and 5–10 reads (checking credits, retrieving history). This translates to less than \$0.01 per user per month in database costs (Firebase, 2025).

The API Pricing & Utilization Table that shows the cost of each api used in the proposed platform

Table 2

API Pricing & Utilization Table

Service Provider	Functionality	Pricing Model	Unit Cost	Thesis Usage Tier
Mobula API	Market Data (Price, Volume)	Freemium (Compute Units)	\$0.00 / scan	Free Tier (100M CU/mo)
GoPlus API	Security Checks (Honeypot)	Free (Rate-Limited)	\$0.00 / scan	Standard Tier (<30 req/min)
Helius API	Solana RPC Connection	Subscription	Included in Fixed Cost	Developer Plan
Groq (Llama 3)	AI Risk Analysis Narrative	Per Token	~\$0.08 / report	On-Demand (Pay-As-You-Go)
Firebase	Database (Firestore)	Usage-Based	\$0.18 / 100k writes	Blaze Plan (Pay-as-you-go)

Service Provider	Functionality	Pricing Model	Unit Cost	Thesis Usage Tier
Netlify	Hosting & Serverless	Flat Monthly Fee	\$19.00 / month	Pro Plan (2M invocations)

Source: Compiled by the author

1.5 Justification of Selected Development Tools

To make the Tokenomics Lab platform functional, it was decided to use an “serverless-first” technology stack. This choice was driven by non-functional requirements for high performance, data integrity, and fast system scalability independent of physical servers. For building the general application structure, it was decided that the primary toolkit should be Next.js 16. This choice was necessary for this project, as it needed parallel access to six different external APIs. Doing this on the client-side natively would overload the browser’s networking stack with many requests, creating performance lag. This problem is alleviated for Next.js via its Server Components feature, allowing for handling heavy operations on the server-side while providing an already-prepared interface, all while keeping safe the handling of API keys. For code integrity contributions, it was decided that the programming environment should use TypeScript 5.9, being integrated well with Next.js. As known, for example, “a Token” model that has such-and-such structure on Solana does have completely different structure properties on Ethereum; hence, JavaScript coding will be extremely risk-prone for successful execution. Strict data typing for security against structural inconsistencies is ensured by using TypeScript. Such inconsistencies will now be handled during the initial build stage instead. This improves the stability of interaction conditions for the user with the general application.

Google Firebase was chosen to handle the back-end functions and the storage of the databases. This was due to the flexibility in the decision to use Firestore, given that different blockchains provide different kinds of metadata; hence, using a SQL database would require different adjustments for each different blockchain introduced into the system. This is addressed by the inherent storage capacity of different data structures by Firestore. Moreover, Firebase has the capacity to handle user authentication and scaling with the number of users.

Regarding cognitive analysis, the Groq SDK with the Llama 3.3 70B model is utilized. The main reason why Groq is considered better than the alternatives with regard to speed is the requirement mentioned in the project specifications to perform a risk analysis within 30 seconds. The hardware Groq uses for its architecture enables inference speed that is significantly faster than the rest of the cloud services, making it possible to create human-readable risk explanations.

The system was built using the Visual Studio Code editor, which was chosen for its rich extension library and native support for TypeScript. To speed up the development of complex functionalities, Amazon kiro acted as the main AI code assistant. In addition to the standard auto-complete abilities of the code assistant, Amazon kiro offered in-depth support, which was adapted to the cloud infrastructure of the respective project. The AI code assistant helped in generating boilerplate code for serverless API routes, helped in debugging the complex code logic of data normalization using the chat feature of the code assistant, and conducted real-time security scanning of code before deployment to detect vulnerabilities. Finally, the infrastructure is hosted in production by Netlify, together with Git and GitHub as a version control system. This setup allows a seamless, fully automated CI/CD process. GitHub is essentially the single ‘source of truth’ for code, storing a safe and secure history of all modifications ever made. Any changes pushed to the central code repository are automatically picked up by Netlify, which then rebuilds a new copy of the Next.js app and pushes it out to a global Edge Network.

APPROVED

Supervisor: Prof.,Paulius Danelius

Bachelor student: Nayan Joshy M.J

Date: 22/12/2025

2. TECHNICAL TASK

1. TITLE OF THE WORK (TOPIC):

A SYSTEM FOR DETECTING CRYPTOCURRENCY TOKEN SCAMS

2. CONTENT OF ANALYTICAL AND RESEARCH WORK:

- 1.1. Analysis of the General Object Characteristics
 - 1.1.1. Tokenomics concept and risk factors
 - 1.1.2. Understanding common types of crypto scams
 - 1.1.3. Case studies of major meme coin incidents
- 1.2. Analysis of functions, Information flows and existing solutions.
 - 1.2.1. Shortcomings and Discrepancies of Existing Solutions
 - 1.2.2. Market analysis of existing solutions
- 1.3. Data Architecture and GDPR Compliance Strategy
 - 1.3.1. Data Collection and Minimization Policy
 - 1.3.2. Technical Implementation of User Rights
 - 1.3.3. Data Security and Processor Management

3. FUNCTIONS OF THE DESIGNED SYSTEM:

- 3.1. Input of contract addresses to trigger automatic chain detection and multi-source data retrieval.
- 3.2. Execution of parallel API calls and normalization of disparate data via an Adapter Pattern.
- 3.3. Execution of a weighted 10-factor algorithm with dynamic overrides for special token types to generate a unified Risk Score.
- 3.4. Integration of Large Language Models to perform Meme Token Classification and generate human-readable risk summaries.
- 3.5. Batch processing functionality allowing users to analyze multiple tokens from a connected wallet.
- 3.6. Implementation of Role-Based Access Control to manage Free, pay-per-use Premium tiers.

4. SYSTEM DESCRIPTION DOCUMENTS AND INSTRUCTIONS:

- 4.1. System User Manual.
- 4.2. Administrator & Developer's Guide
- 4.3. System structure overview and architectural diagrams.

5. SYSTEM DESIGN TOOLS, SOFTWARE AND HARDWARE REQUIREMENTS:

- 5.1. Design and logic modelling tools: Draw.io, V0 , GitHub Copilot, Gemini 3.
- 5.2. Frontend and core logic tools: Next.js 16, Node.js 20.x , TypeScript 5.9 , Tailwind CSS, Schaden ui.
- 5.3. Backend: Firebase Blaze plan.
- 5.4. Development tools: Visual Studio Code, Git , Postman, GitHub Copilot, Gemini 3 pro, Claude 4.5 Sonnet, Amazon Kiro.

- 5.5. DB and Log control system: Google Firebase Authentication, Firestore.
- 5.6. AI and External APIs: Groq SDK, Mobula API, GoPlus Security API, x402 Protocol (Payments), Helius API, Moralis API, Coin Gecko API.
- 5.7. System operating environment: Platform-agnostic web environment (accessible via Chrome, Edge, Safari); Development on Windows 10+/MacOS/Linux.
- 5.8. Technical equipment requirements: Standard development workstation equipped with at least AMD Ryzen 5/M1 or equivalent, 8GB RAM, stable internet connection for API data streaming.
- 5.9. **AI tools for content structuring, research, and grammar verification:** Gemini 3 Pro, Grammarly, Perplexity AI, Grok.
- 6. **INFORMATION SYSTEM TESTING AND EVALUATION:**
 - 6.1. Evaluation through simulated attack scenarios to validate system responses.
 - 6.2. Beta user testing
 - 6.3. Manual functional testing of the payment system (x402 Protocol) via wallet transfers of test net assets to verify transaction detection and automated premium tier unlocking
 - 6.4. Evaluation of the system's detection capabilities and AI response accuracy.
- 7. **WORK DELIVERY REQUIREMENTS:**
 - 7.1. Comprehensive thesis following the provided methodological guidelines.
 - 7.2. Oral presentation and defense (6-8 minutes) supported by a PowerPoint presentation (pptx format).

3. PROJECT PART

Having discussed the theoretical aspects of the proposed system in the analytical part the project part will move forward with designing the whole system in a feasible manner.

This chapter will discuss the architecture of the proposed solution, the rationale for the choice of the technology stack, and the internal logic of the essential components, namely the 10-Factor Risk Calculation Engine and the Chain-Adaptive Data Fetcher. This chapter will also elaborate on data modelling and defend the security mechanisms developed for GDPR compliance.

3.1 Purpose of the project

The main goal of the Tokenomics Lab platform is to help everyday investors who are currently at a disadvantage in the decentralized finance market. As discussed in the analysis chapter, thousands of new tokens are launched every day. This creates a confusing environment where it is extremely difficult for non-technical users to spot the difference between a real project and a scam, such as a "rug pull."

To solve this, the system was built to be a unified automation tool. Its purpose is to replace the slow, manual process where a user currently must check three or four different websites (like block explorers and trading charts) to verify a token. Instead, this platform does all that work instantly. It pulls raw data from security and market APIs and calculates a single Risk Score, providing a standard way to measure safety whether the token is on Ethereum or Solana.

Most importantly, the system does more than just show numbers. By integrating the Groq AI engine, it translates confusing technical data into plain English. This ensures that the user doesn't just see a "High Risk" warning but understands why the token is dangerous. Ultimately, the project aims to give individual traders the same kind of safety tools that professional institutions use, helping them avoid bad investments in a risky market.

3.2 Logical Structure of the System

To ensure modularity, scalability, and ease of maintenance, the logical structure of the Tokenomics Lab platform follows a multi-tiered Service-Oriented Architecture. This approach separates the user interface from the complex risk calculation logic and data persistence layers, allowing each component to evolve independently.

The system is conceptually organized into four distinct layers.

Table 3

Logical structure of the system layers and components

Layer Name	Components & Technology	Primary Responsibilities
Presentation Layer	Next.js 16, React 19, Tailwind CSS, Recharts, Framer Motion	Captures user input and renders the visual dashboard. Utilizing Server Components ensures sensitive logic remains hidden from the browser.
Application Service Layer	API Routes, Risk Calculator, Subscription Manager, Rate Limiter	Manages the 10-Factor Risk Algorithm, enforces subscription limits, and handles Rate Limiting before requests reach external APIs.
Cache & Optimization Layer	In-Memory Cache	Intercepts requests to return cached risk reports for popular tokens, significantly reducing external API costs and ensuring fast response times.
Data Persistence Layer	Firebase Auth, Firestore (NoSQL), Storage	Manages persistent records including User Identities, Watchlists, Analysis History, and Admin Activity Logs. Secured via Firestore Security Rules.

Source: Compiled by the Author.

3.3 Functional Requirements

The table below illustrates the functional requirements of the Platform.

Table 4

Functional requirements of the proposed system

ID	Requirement	Description	Priority	Component
FR-01	Token Risk Analysis	Multi-source data aggregation , 10-factor algorithm , AI enhancement , Risk classification	P0	Risk Calculator + AI Service
FR-02	User Authentication	Email/OAuth registration, Firebase Auth, tier-based access control	P0	Auth Context + Firebase
FR-03	Interactive Dashboard	Risk visualization with charts, factor breakdown, AI accordion	P0	Presentation Layer
FR-04	Subscription Management	3-tier model	P0	Rate Limiter + Profile
FR-05	Watchlist Management	Save tokens, monitor changes, trigger alerts (Premium)	P1	Watchlist + Alert Service
FR-06	Multi-Chain Support	Ethereum, BSC, Solana, Polygon, Cardano with adaptive weights	P0	Chain-Adaptive Fetcher
FR-07	Admin Panel	User management, analytics, activity logs, system settings	P1	Admin Module
FR-08	2FA Security	TOTP-based two-factor authentication for admins	P1	Security Layer

ID	Requirement	Description	Priority	Component
FR-09	AI Analysis	Meme detection + comprehensive summary generation	P1	Groq/Gemini AI
FR-10	Activity Logging	Audit trail for all user actions with IP/timestamp	P2	Activity Logger
FR-11	Data Export	GDPR-compliant data export	P2	User Service
FR-12	Account Deletion	Permanent account + data removal	P2	User Service
FR-13	Crypto Payments	Accept SOL/USDC payments via x402 integration to upgrade users to Premium tier automatically.	P1	Payment Gateway (x402)

Source: Compiled by the Author.

3.4 Non-Functional Requirements

The table below illustrates the Non-Functional requirements of the proposed system.

Table 5

Non-Functional requirements of the proposed system

ID	Category	Requirement	Target Metric	Enforcement
NFR-01	Performance	Fast response times	<30s fresh, <5s cached	Caching + Optimization
NFR-02	Security	Server-side logic hiding	100% API routes	Next.js Server Components
NFR-03	Consistency	Unified data format	0 schema violations	Adapter Pattern
NFR-04	Scalability	Rate limiting enforcement	100% compliance	Firestore counters

ID	Category	Requirement	Target Metric	Enforcement
NFR-05	Availability	Fallback mechanisms	99.9% uptime	Multi-source failover
NFR-06	Efficiency	Intelligent caching	>50% cache hit rate	TTL strategy (1-30min)
NFR-07	Maintainability	Code quality	0 TSC errors	TypeScript strict mode
NFR-08	Usability	User experience	LCP <2.5s	Mobile-first design
NFR-09	Reliability	System stability	<0.1% error rate	Error logging + backups
NFR-10	Privacy	GDPR compliance	100% compliant	Data export/deletion
NFR-11	Resilience	API fault tolerance	<1% API failures	Retry logic + circuit breakers
NFR-12	Observability	Full monitoring	All metrics tracked	Firebase Analytics + Admin analytics

Source: Compiled by the Author.

3.5 Use case diagram of the Tokenomics Lab Platform

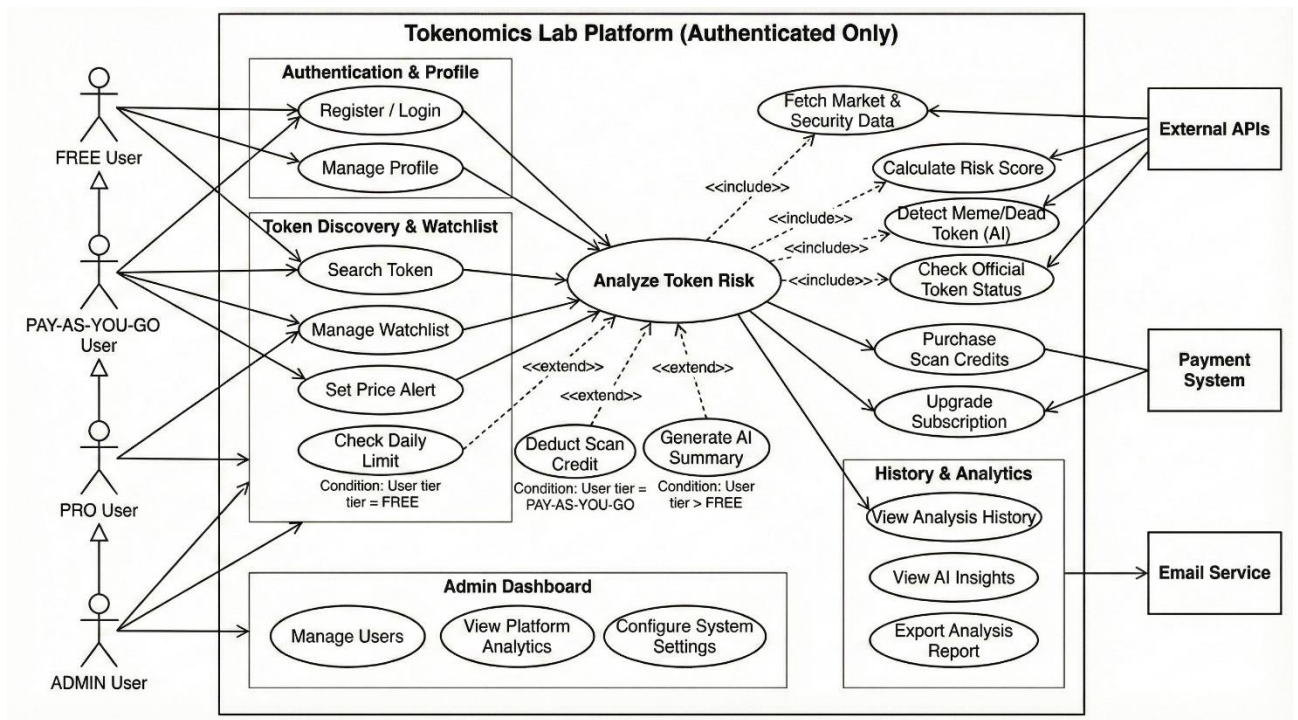


Figure 3: Use case diagram of Tokenomics lab

Source: Compiled by the author with the help of gemini 3.

Figure 3 visualizes the user journey within the platform, mapping out exactly who can do what. I designed the system with a strict "Authenticated Only" boundary, meaning there is no guest access because features like watchlists and credit balances require a persistent user profile.

The User Hierarchy (Actors): Instead of treating every user type as a separate entity, Actors were organized into a hierarchy to show how permissions stack up:

- **FREE User:** This is the baseline. They can register, manage their profile, and perform standard risk analysis (capped at 20 scans/day).
- **PAY-AS-YOU-GO User:** This actor builds upon the Free tier. They have all the same basic rights but gain the specific ability to Purchase Scan Credits. This separation allows me to model the micropayment flow without complicating the base user role.
- **PREMIUM User:** This is the power user. They inherit everything from the tiers below but gain exclusive access to long-term memory features like History & Analytics and unlimited Watchlists.

- **ADMIN User:** Finally, the Admin has the most authority. They can do everything a Premium user can do, plus they have exclusive access to the Admin Dashboard to manage users or view platform revenue etc.

The Core Analysis Logic: Analyse Token Risk, is the heart of the system as a composite process to show that it isn't just one simple action:

- **Mandatory Steps (<<include>>):** You cannot calculate a risk score without data. That is why <<include>> arrows were used to connect Fetch Market Data and Calculate Risk Score. These happen every single time a user runs a scan.
- **Optional Steps (<<extend>>):** Features like **Generate AI Summary** or **Deduct Scan Credit** don't happen every time. They only trigger under specific conditions. <<extend>> arrows were used to show that these are conditional add-ons to the main analysis flow.

External Systems: The diagram also highlights that the platform doesn't exist in a vacuum. It actively relies on three external forces: External APIs (like Mobula and GoPlus) for raw data, x402 for payments, and an Email Service for sending out alerts.

3.6 Computerised system workflow

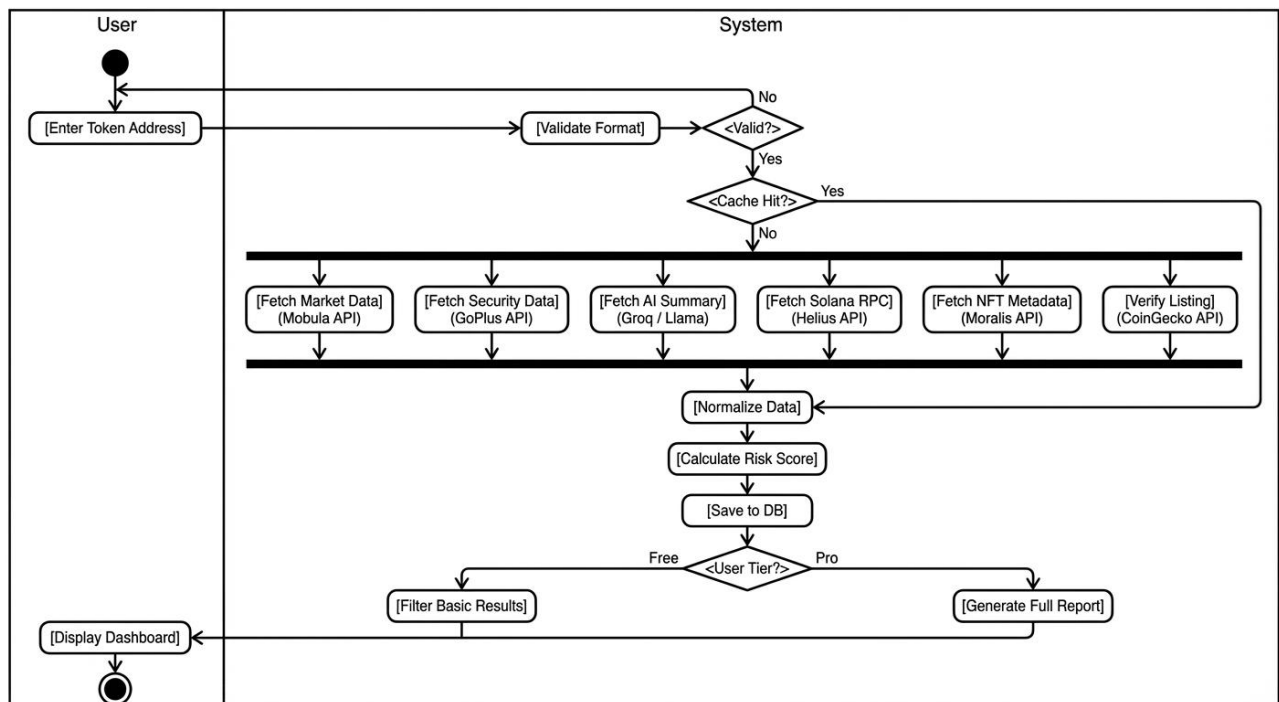


Figure 4:Activity Diagram of the system

Source: Compiled by the Author.

Figure 4 breaks down the step-by-step logic of the token analysis engine. By splitting the diagram into **User** and **System** swim lanes, The diagram maps out exactly how a simple user request triggers a complex series of backend operations.

The workflow highlights three key engineering decisions that was made during implementation:

- **Validation & Cache Strategy:** Efficiency was the top priority. As the diagram shows, the system doesn't just blindly fetch data; it first validates the contract format and then checks the internal Redis cache. If a Cache hit occurs, the system shortcuts the entire process, skipping the heavy API calls and delivering instant results. This logic prevents redundant processing for popular tokens.
- **Parallel Data Fetching (Fork/Join Pattern):** When new data *is* needed, speed is critical. Instead of calling APIs one by one the Fork/Join pattern was implemented as shown in the centre of the diagram. This allows the system to fire off six simultaneous requests to Mobula, GoPlus, Groq, and others at the exact same time. This parallel approach reduced the total wait time from over 12 seconds down to just 2-3 seconds.
- **Tier-Based Filtering:** Finally, the workflow enforces the business model. After the risk score is calculated, the system checks the user's subscription tier. As depicted, **Free** users

are routed to a limited "Basic" view, while **Pro** users unlock the full, detailed report. This ensures that premium features are securely gated at the logic level, not just hidden on the frontend.

3.7 Hierarchy Of Computerized Functions

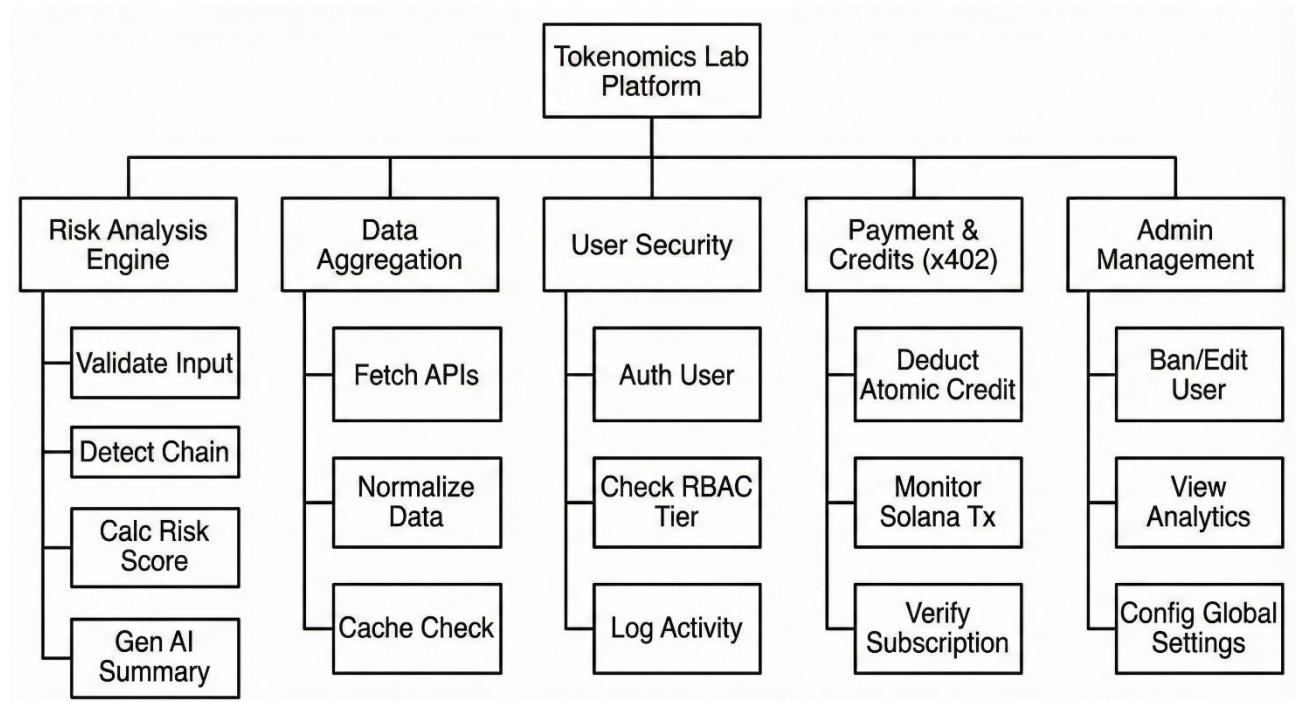


Figure 5:Platform Hierarchy Diagram.

Source: Compiled by the Author.

To manage the system's complexity effectively, the architecture was decoupled into five distinct subsystems. This modular design ensures that a failure in one component does not compromise the entire application. As visualized in Figure 5, the functional hierarchy is organized as follows:

1. Risk Analysis Engine This subsystem acts as the platform's core processing unit, converting raw data into actionable intelligence.

- **Validate Input:** Serves as the primary security gatekeeper. It rigorously sanitizes user input to prevent injection attacks and distinguishes between address formats.
- **Detect Chain:** Automates network identification to enhance usability. Instead of requiring manual selection, this module tests RPC connectivity to automatically resolve the target blockchain.

- **Calc Risk Score:** Executes the mathematical core of the platform. It runs the 10-Factor Algorithm, applying dynamic weights such as prioritizing "Liquidity Depth" on Solana versus "Tax Settings" on BSC to compute the final 0–100 risk score.
- **Gen AI Summary:** Manages the cognitive layer. This module packages technical findings into a prompt context and handles asynchronous communication with the Groq SDK to generate human-readable narratives.

2. Data Aggregation This module manages the ingestion and standardization of external data, ensuring the analysis engine receives accurate inputs.

- **Fetch APIs:** Designed for high-speed performance, this function executes parallel requests to external providers (Mobula, GoPlus, Helius), ensuring total analysis time remains under 3 seconds.
- **Normalize Data:** Acts as a "universal translator," mapping divergent JSON schemas from various providers into a single, unified CompleteTokenData object.
- **Cache Check:** optimizes resource usage by querying the Redis/In-Memory cache for recent reports before triggering costly external API calls.

3. User Security This subsystem enforces access policies and secures user data.

- **Auth User:** Blocks unauthorized access by managing session tokens via Firebase Auth, supporting both Google OAuth and standard email/password authentication.
- **Check RBAC Tier:** Implements Role-Based Access Control (RBAC) by dynamically verifying the user's tier (Free, Pay-Per-Use, or Premium) in real-time to gate access to sensitive features.
- **Log Activity:** Ensures compliance and security by creating an immutable audit trail, logging critical actions such as logins and payment events to the database.

4. Payment & Credits (x402) This module manages the hybrid monetization model and integrates with the x402 payment protocol.

- **Deduct Atomic Credit:** Handles the internal economy by safely decrementing credit balances in Firestore. It uses strict database locking to prevent "double-spending" during high-traffic periods.

3.8 Data flow diagram of computerised system

To illustrate how information moves through the Tokenomics Lab platform, the data flow is modelled at two levels the Context Level (Level 0) and the Functional Decomposition Level 1.

3.8.1 Context Diagram (Level 0 DFD)

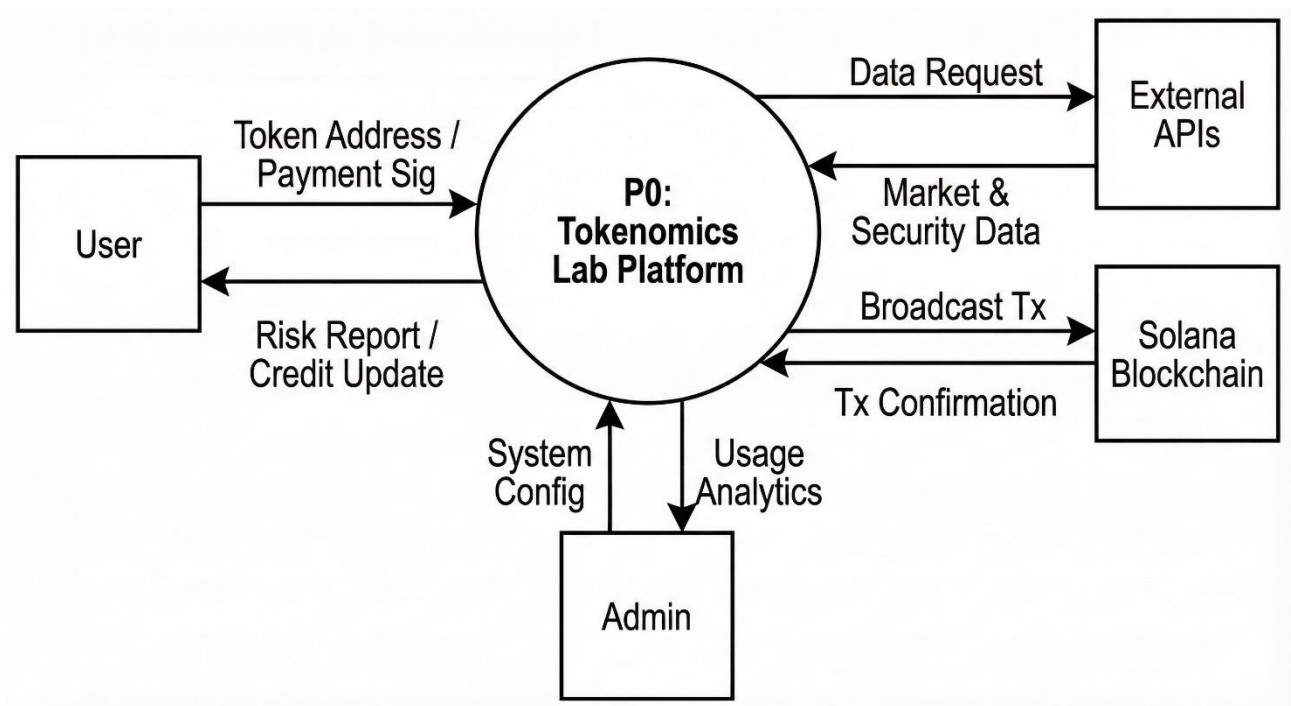


Figure 6:Context diagram

Source: Compiled by the Author.

In **Figure 6**, The entire Tokenomics Lab is modelled as a single, high-level process (P0). This view is designed to define the system's boundaries, treating the internal logic as a "black box" to focus strictly on what information enters and leaves the application.

- **The User Interaction:** The primary data flow begins with the User, who sends two critical types of input: raw search queries (Token Addresses) and financial commitments (Payment Signatures). In return, the system delivers processed Risk Reports and real-time dashboard updates.
- **External Dependencies:** The diagram highlights that the system cannot function in isolation. It must constantly pull raw market JSON from External APIs (Mobula, Helius) to perform its calculations.
- **Blockchain Synchronization:** Unlike a standard web app, this system has a direct, bi-directional relationship with the Solana Blockchain. It broadcasts payment transactions to the network and waits for on-chain confirmation before unlocking features.
- **Administrative Oversight:** Finally, the Admin entity sits outside the core process, injecting configuration data (like pricing changes) that alters how the system behaves for regular users.

- platform as a single, high-level process (P0). This view is designed to define the system's boundaries, treating the internal logic as a "black box" to focus strictly on what information enters and leaves the application.

3.8.2 Token Analysis Data Flow (Level 1)

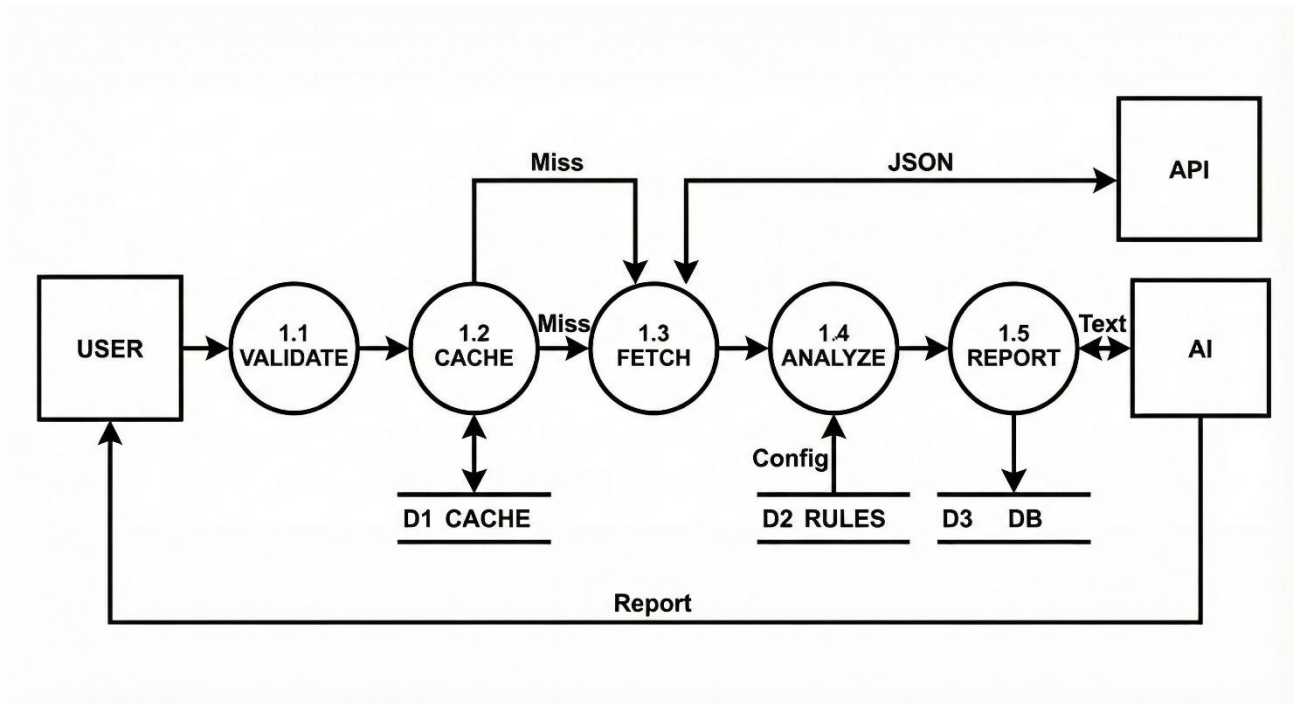


Figure 7:Token Analysis Data Flow (Level 1)

Source: Compiled by the Author.

Figure 7 breaks down the internal logic of the "Token Analysis" feature. While the previous Context Diagram treated the system as a unified black box, this Level 1 diagram exposes the specific data pipeline engineered to balance processing speed with analytical depth.

The architecture consists of five distinct processes, which was structured to minimize unnecessary external calls.

The pipeline begins the moment a user submits a contract address. This validation step was implemented as a strict gatekeeper to reject malformed strings (like non-hexadecimal characters) immediately. This ensures that invalid queries never reach the database, preventing wasted processing power.

As illustrated in the diagram, the system prioritizes local data. This "Cache First" strategy was designed after realizing that fetching data directly from blockchain nodes was introducing unacceptable latency often 2 to 3 seconds per request. If the analysis exists in the D1 Cache Store (a

"Hit"), the system skips the heavy lifting and jumps straight to reporting, which makes the app feel significantly more responsive.

When a "Cache Miss" occurs, the system moves to this aggregation step. Its role is to query multiple External APIs to gather raw security and market data. This was deliberately isolated into its own process to handle API rate limits and potential timeouts gracefully, ensuring that a single failed external request doesn't crash the entire analysis engine.

This process is the brain of the operation. It ingests the raw JSON data and applies the weighting logic stored in D2 Rules. Separating this calculation logic from the data retrieval code was a key design decision; it allowed me to fine-tune risk parameters such as how heavily "low liquidity" impacts the final score without having to touch the fetching mechanism.

Finally, the system compiles the calculated risk score and AI-generated summary into a readable format. This process serves a dual purpose it delivers the real-time report to the User interface and simultaneously archives the result in the D3 DB, enabling the "History" to feature I discussed in the requirements section.

3.8.3 Payment and RBAC Data Flow (Level 1 DFD)

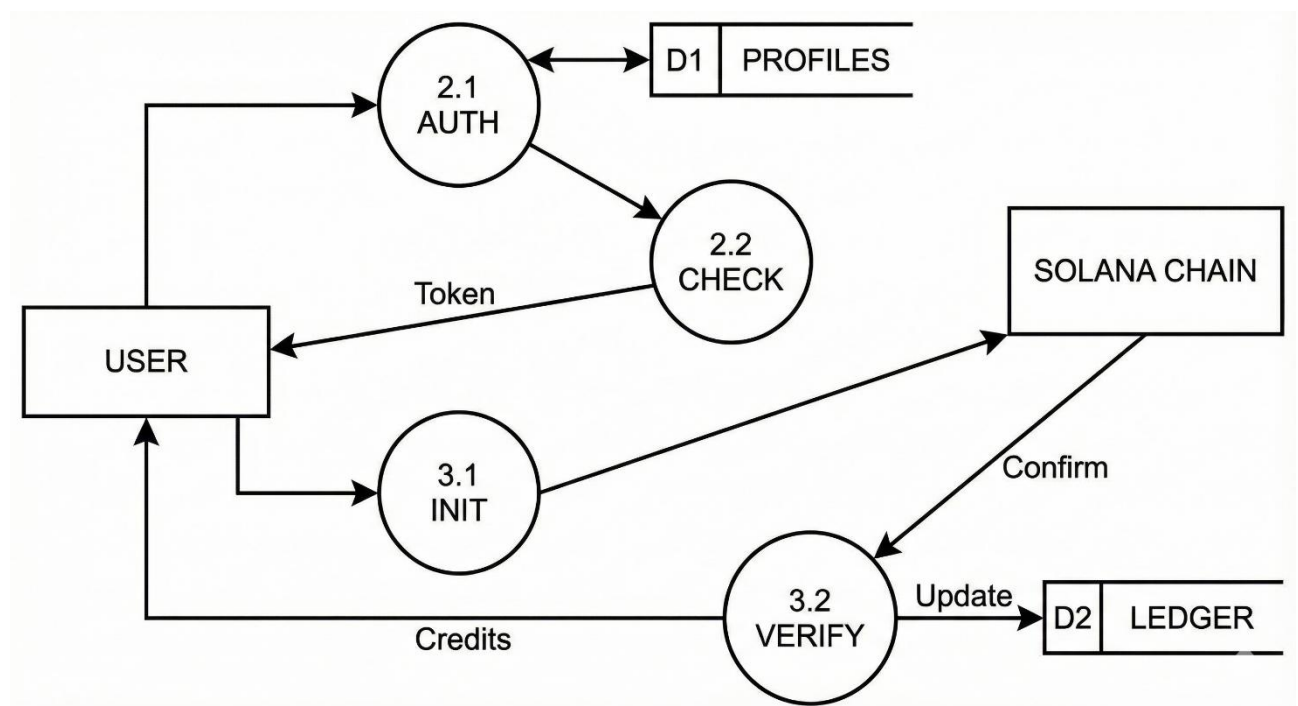


Figure 8:Payment and RBAC Data Flow diagram (Level)

Source: Compiled by the Author.

Figure 8 details the data flows for the platform's two critical support systems: Role-Based Access Control (RBAC) and the x402 Payment Processing engine. Unlike the analysis pipeline,

which focuses on data transformation, these flows are designed around strict transaction integrity and security.

The diagram illustrates two parallel operational streams:

- **Access Control Stream (Top Path):** This flow governs user sessions. When a user logs in, the 2.1 Auth process verifies their credentials against the D1 Profiles database. I designed this as a synchronous check to ensure no unauthorized traffic reaches the inner system. Once validated, the 2.2 Check process reads the user's subscription tier and issues a session token. This token acts as a "passport," allowing the user to access protected routes without constant database re-verification.
- **Payment Stream (Bottom Path):** The payment logic is decoupled to handle the asynchronous nature of blockchain transactions. The flow begins when a user requests credits via 3.1 Init, which constructs a payment intent. Crucially, the actual value transfer happens externally on the Solana Chain. The system's 3.2 Verify process acts purely as a listener; it waits for the on-chain "Confirm" signal before updating the internal D2 Ledger. This design ensures that our internal database never drifts from the on-chain reality, a user is only credited if the blockchain explicitly confirms the funds were received.

3.9 Conceptual object model

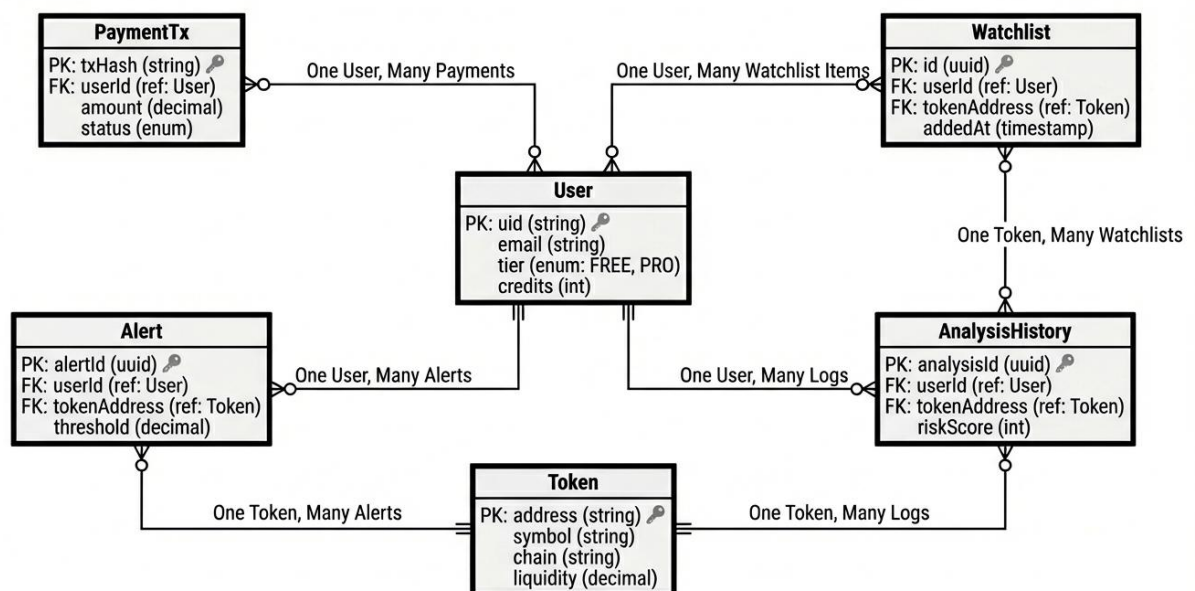


Figure 9:ERD diagram

Source: Compiled by the Author.

Although the physical database implementation utilizes Google Firestore to handle the volatile nature of blockchain data, the logical architecture was designed to follow a strict relational structure. This approach ensures data integrity, particularly for financial transactions and user access control. The logical relationships between these entities are visualized in the Entity-Relationship Diagram (Figure 9).

The model was built around five core entities, which were defined to balance query performance with data consistency:

1. User Entity : The User entity was designed to serve as the central anchor for the entire system. Unlike traditional SQL tables that might split data across many rows, profile settings and authentication metadata were stored directly within this document. This structure reduces the number of database reads required to load the dashboard, significantly improving load times.

- **Key Attributes:** uid (Primary Key), tier (Free/Premium), and credits (Integer).

2. Payment Transaction (The Ledger) To strictly enforce the integrity of the x402 Micropayment protocol, financial data was decoupled from the user profile. An immutable PaymentTx entity was created to act as a "Double-Entry" ledger. This design choice ensures that every single credit added or deducted is traceable back to a specific on-chain Solana signature, preventing any possibility of "double-spending" or fraud.

- **Key Attributes:** txHash (Solana Signature), amount (USDT), and status (Confirmed/Failed).

3. Token Entity (Reference Data): Instead of duplicating static data (like a token's symbol or decimal count) into every single user's watchlist, this data was normalized into a master Token entity. This reference table is identified by the contract address, ensuring that if a token's metadata updates, it reflects instantly across the entire platform.

- **Key Attributes:** address (Contract Address), chain (e.g., "501" for Solana), and liquidity.

4. Analysis History : The Analysis History entity was designed using a "Snapshot" pattern. When a risk scan is performed, the system saves a static copy of the risk score and market data at *that exact moment*. This is critical for the "History" feature because it allows users to see how a token's risk profile has changed over time, rather than just seeing the current live data.

- **Key Attributes:** riskScore (0-100) and aiSummary (Text).

5. Watchlist & Alerts: To optimize dashboard performance, Watchlist and Alert items were modeled as dependent children of the User. By nesting these references, the system can fetch a user's entire portfolio in a single database query without needing complex and slow "JOIN" operations.

3.10 System states and processes

The platform is organised around a clear lifecycle for each analysis request so that its behaviour remains predictable, testable, and easy to debug during iterative Scrum sprints. In practice, this means that every request passes through a small set of well-defined states, and each transition between states is triggered by a specific process rather than by ad hoc logic scattered across the codebase. This structure supports reliable logging, ethical auditing of user actions, and reproducible evaluation, all of which are emphasised in the methodological guidelines for software projects.

3.10.1 Core system states

An analysis request begins its life in a simple waiting state and then moves step by step toward either a successful result or a controlled failure.

Each state has a clear purpose:

- **Pending:** As soon as a user submits a token or address, the request is created and marked as pending. At this point, nothing has been fetched or computed yet; the system has only acknowledged that the request exists.
- **Validating:** The request is then checked against the platform's rules. In this state, the system verifies the user's access tier (Free, Pay-As-You-Go, or free), enforces daily limits or credit balances, and confirms that the input itself is well formed. If a free user has already reached the daily scan limit or credits are missing for a paid feature, the request does not proceed further and is rejected.
- **Fetching:** When validation succeeds, the request moves into the fetching state. Here, the Chain-Adaptive Fetcher decides which external services to call based on the blockchain in question, combining security checks and market data in a way that minimises cost and latency. Non-critical data sources are treated as optional so that a temporary outage does not automatically invalidate the whole analysis.
- **Analyzing:** Once the required data has been collected, the request enters the analysing state. For basic scans, this might mean computing a risk score or flagging obvious red signals using deterministic rules. For premium features, this is also the point where richer outputs such as AI-generated risk narratives are produced, building on the previously fetched data.
- **Completed:** When all calculations are finished and the response is ready, the request is marked as completed. The result is stored in the database and returned to the client, so it

can later be included in user histories, evaluation metrics, or anonymised studies of usage patterns in line with GDPR-compliant research practices.

- **Failed:** If something goes wrong at any step invalid input, permission problems, external API timeouts, or unexpected errors the request is moved into a failed state rather than left hanging. This ensures that every unsuccessful attempt has a clear end point and can be logged for later inspection, which is essential for systematic bug fixing and sprint retrospectives.

3.10.2 Processes and transitions

The path from the initial submission to a completed or failed outcome is governed by a small set of repeatable processes. These processes define how the system reacts to user actions and external dependencies, and they give a concrete basis for designing tests, monitoring, and future extensions.

Request initiation: When a user triggers a scan from the mobile interface or web frontend, a new request object is created and immediately placed in the pending state. This step is intentionally lightweight to keep the user interface responsive, especially under higher loads handled by serverless infrastructure.

Pre-flight validation: The system then performs a quick validation. It checks which tier the user belongs to, how many scans have already been performed that day, and whether there are enough credits for any premium features requested. If these checks fail, the request is moved straight to the failed state with a clear error code, so that the client can display a meaningful message instead of a generic error.

Data aggregation: For requests that pass validation, the next step is data aggregation. In this process, the Chain-Adaptive Fetcher decides which APIs to call such as security services for contract risk checks and market data providers for pricing and volume

3.10.3 Formal description of calculations

The platform relies on a weighted multi-factor risk scoring model, combined with a small set of post-processing rules, to turn raw on-chain and off-chain data into a single token risk score on a 0–100 scale. This structure mirrors common practice in crypto risk assessment, where multiple dimensions such as liquidity, concentration, and contract permissions are aggregated into a composite indicator for easier interpretation. In addition, the same section formalises how user credits, pay-as-you-go payments, and daily limits are computed, so that all numerical logic in the system remains transparent and reproducible.

3.10.4 10-Factor Risk calculation Breakdown

Table 6

Table of Risk factors and calculations

Factor ID	Name (Weight)	Computation/Thresholds
F ₁	Supply Dilution (18%)	FDV/MCAP ratio scoring: - 10 points: $\leq 1x$ (minimal dilution) - 30 points: $\leq 2x$ - 50 points: $\leq 5x$ - 70 points: $\leq 10x$ - 90 points: $> 10x$ (severe dilution) Penalties: +20 if mintable; +15 if unlimited supply with no burns
F ₂	Holder Concentration (20%)	Top 10 holders percentage: - 10 points: $< 20\%$ (excellent distribution) - 25 points: 20-39% - 50 points: 40-59% - 75 points: 60-79% - 95 points: $\geq 80\%$ (extreme whale dominance)
F ₃	Liquidity Depth (16%)	FDV/Liquidity ratio scoring: - 5 points: ≤ 3 (deep liquidity) - 20 points: 3-6.67 - 40 points: 6.68-12.5 - 65 points: 12.6-33.3 - 90 points: > 33.3 (thin liquidity, high slippage risk)
F ₄	Vesting Unlock (13%)	Upcoming token unlocks (next 30 days): - 10 points: $< 5\%$ supply unlocking - 30 points: 5-9% unlocking - 50 points: 10-19% unlocking - 70 points: 20-29% unlocking - 90 points: $\geq 30\%$ unlocking (critical flood risk) Note: Not yet implemented due to vesting data availability constraints
F ₅	Contract Control (15%)	Additive penalty system: - +40 points: Mint function enabled - +30 points: Honeypot detected - +30 points: Owner not renounced - +25 points: Freeze authority (Solana)
F ₆	Tax/Fee (11%)	Total buy + sell tax: - 5 points: 0% tax - 20 points: $\leq 5\%$ - 40 points: $\leq 10\%$

Factor ID	Name (Weight)	Computation/Thresholds
		- 70 points: $\leq 20\%$ - 95 points: $> 20\%$
F ₇	Distribution (8%)	Dual metric system: Team Allocation: - +15 points: $> 20\%$ team allocation - +25 points: $> 30\%$ team allocation - +35 points: $> 40\%$ team allocation Top-10 Concentration: - +8 points: $\geq 30\%$ - +15 points: $\geq 40\%$ - +25 points: $\geq 50\%$ - +35 points: $\geq 60\%$ - +45 points: $\geq 70\%$ - +55 points: $\geq 80\%$
F ₈	Burn/Deflation (6%)	Percentage of supply burned: - 10 points: $\geq 50\%$ burned - 30 points: 20-49% - 50 points: 5-19% - 70 points: $< 5\%$ burned
F ₉	Adoption (10%)	24-hour transaction count: - 5 points: $\geq 10,000$ transactions - 20 points: 1,000-9,999 - 45 points: 100-999 - 70 points: 10-99 - 90 points: < 10 transactions
F ₁₀	Audit Transparency (4%)	GoPlus API integration: With GoPlus data: - +50 points: Contract not open source - +30 points: Liquidity not locked Fallback (no GoPlus): - Base: 60 points (moderate risk) - +20 points: Market Cap/Liquidity ratio > 100

Source: Compiled by the user with the help of Perplexity (Joshy N. J., 2025)

3.10.5 Chain-Adaptive Weights and Overrides

Weights shift strategically depending on the blockchain to better capture each ecosystem's unique risk characteristics. All weight profiles are automatically normalized to ensure they total exactly 100%, preventing score inflation and maintaining mathematical accuracy.

Table 7

Chain-Adaptive Weight Distribution by Ecosystem

Risk Factor	Standard EVM Chains (ETH, BSC, Polygon, ARB, OP, Base)	Solana Network	Meme Token Profile (All Chains)
Holder Concentration	20%	18.5%	24%
Supply Dilution	18%	12%	14%
Liquidity Depth	16%	16.7%	20%
Contract Control	15%	32.4%	12%
Tax/Fee	11%	0%	10%
Adoption	10%	9.3%	15%
Distribution	8%	5.6%	6%
Burn/Deflation	6%	3.7%	2%
Audit Transparency	4%	1.9%	1%

Source: Compiled by the Author.

Raw totals may exceed 100% before normalization. The system automatically scales all weights proportionally to ensure mathematical validity (e.g., Standard EVM weights are each multiplied by $100/108 = 0.926$ to achieve exact 100% total).

Normalized Weight Formulas:

For each chain profile, the normalization process applies:

$$W_{normalized}^i = \frac{W_{base}^i}{\sum_{j=1}^{10} W_{base}^j}$$

where W_{base}^i represents the base weight for factor i and the denominator ensures $\sum W_{normalized} = 1.0$.

3.10.6 Post-Calculation Overrides

Targeted overrides fine-tune results for edge cases:

- Official Tokens: Risk reduction based on verification status verified non-meme tokens receive -15 points, verified meme tokens receive -10 points, and verified tokens exceeding \$1B market capitalization receive -25 points.
- Dead Tokens: When ≥ 2 conditions are met (Liquidity $< \$100$, Volume $< \$10$, Transactions = 0, or 30-day price drop $> 95\%$), the score is overridden to 95 points (CRITICAL).
- Meme Tokens: A +15 point penalty applies when AI classification confidence exceeds 70%, accounting for inherent volatility and speculation risk.
- Stablecoins: Major stablecoins like USDT, USDC, DAI, FRAX receive automatic override to 10 points (LOW risk) regardless of other factors.

3.10.7 Final Score Calculation and Risk Bands

The composite risk score is computed using a weighted summation of individual factor scores, adjusted by chain-adaptive weights and post-calculation overrides.

The mathematical formulation is expressed as:

$$S_{final} = \text{clamp} \left(\sum_{i=1}^{10} F_i \times W_i + O, 0, 100 \right)$$

where:

- S_{final} represents the final risk score
- F_i denotes the individual score for factor i (where $i \in \{1, 2, \dots, 10\}$)
- W_i represents the normalized chain-adaptive weight for factor i (where $\sum_{i=1}^{10} W_i = 1.0$)
- O denotes the cumulative override adjustments (positive or negative)
- $\text{clamp}(x, a, b)$ constrains the value x to the interval $[a, b]$

For example, consider a hypothetical EVM token analysis:

$$S_{final} = \text{clamp}(70 \times 0.20 + 50 \times 0.18 + 40 \times 0.16 + 30 \times 0.15 + 20 \times 0.11 + 45 \times 0.10 \\ + 35 \times 0.08 + 25 \times 0.06 + 60 \times 0.10 + 40 \times 0.04 + 0, 0, 100)$$

Calculating step by step:

- Holder Concentration: $70 \times 0.20 = 14.0$

- Supply Dilution: $50 \times 0.18 = 9.0$
- Liquidity Depth: $40 \times 0.16 = 6.4$
- Contract Control: $30 \times 0.15 = 4.5$
- Tax/Fee: $20 \times 0.11 = 2.2$
- Adoption: $45 \times 0.10 = 4.5$
- Distribution: $35 \times 0.08 = 2.8$
- Burn/Deflation: $25 \times 0.06 = 1.5$
- Vesting Unlock: $60 \times 0.10 = 6.0$
- Audit Transparency: $40 \times 0.04 = 1.6$

$$S_{final} = \text{clamp}(14.0 + 9.0 + 6.4 + 4.5 + 2.2 + 4.5 + 2.8 + 1.5 + 6.0 + 1.6 + 0,0,100) = 52.5$$

This yields a final score of 52.5, placing the token in the MEDIUM risk band.

The automatic normalization process ensures all weight profiles total exactly 100%, preventing mathematical errors and score inflation that could occur with improperly balanced weights. This was calculated and compiled with the assistance of perplexity (Joshy N. J., 2025).

Table 8

Table of Risk Bands

Score Range	Classification	Recommendation
0-30	LOW	Generally safe for standard usage with routine vigilance
31-60	MEDIUM	Proceed with caution; consider smaller allocations and seek corroborating fundamentals
61-80	HIGH	Significant risks; suitable only for highly risk-tolerant users with strong justifications
81-100	CRITICAL	Avoid; indicators strongly resemble scam patterns or structurally unsound tokenomics

Source: Compiled by the Author.

4. PART OF SOFTWARE IMPLEMENTATION

This chapter dives into the details of how the software was gradually built, covering its Firebase database structure, key UI and backend modules, their interactions, testing examples, and deployment guides for full reproducibility

4.1 Implementation environment

The platform runs in a modern, serverless stack optimized for scalability and low student-budget costs, developed iteratively across Scrum sprints on windows 11 with VS Code. This setup leverages cloud-native services for deployment without managing servers, aligning with GDPR data handling via Firebase's anonymization tools

Table 9

Table of Technology Stack

Component	Technology/Version	Purpose
Frontend	Next.js 14 (React 18)	UI modules, SSR for SEO
Backend	Next.js API routes (serverless)	Core logic, Chain-Adaptive Fetcher
Database	Firebase Firestore (Blaze plan)	User data, logs
Auth	Firebase Auth	Role-based tiers
Hosting	Netlify Pro (\$19/mo)	Global CDN, 2M functions/mo

Source: Compiled by the Author.

4.2 Database Specification

Tokenomics Lab uses Firebase Firestore as its sole data store, chosen for serverless scaling, real-time sync, and seamless Next.js integration. This hierarchical model avoids relational joins, suiting denormalized scan results and user-scoped queries, with security rules enforcing GDPR-compliant access.

The data model is built around a single, central concept: each user has their own “slice” of the database, and everything else is attached to that slice. The Users collection holds the basics identity, tier, and daily scan counters while the Credit System tables record how many credits the user has and

how those credits change over time, which ties directly into the pay-as-you-go and Premium business model. Around this core, the Watchlist, Analysis History, and Alert Notifications tables capture what the user is interested in: which tokens they follow, what risk analyses have been run, and which alerts were triggered when risk or price moved in a worrying way. Finally, the Activity Logs table provides a clean audit trail of important actions like logins, scans, upgrades, and data exports, so behaviour can be monitored and debugged without touching the infrastructure. Taken together, these tables form a user-centred, traceable data layer that supports both day-to-day app features and longer-term evaluation of how the system is used.

4.2.1 Users Collection Schema

Table 10

Table of Users Collection

Field	Type	Description	Constraints
uid	string	Firebase Auth UID	PK, immutable
email	string	User email	Unique, indexed
displayName	string	Display name	Max 100 chars
tier	enum	Access tier	FREE PAYGO PRO ADMIN
createdAt	Timestamp	Creation date	Auto-set
tokensAnalyzed	number	Daily scans (FREE)	UTC reset
lastScanDate	Timestamp	Last activity	Rate limit

Source: Compiled by the Author.

4.2.2 Watchlist Sub-Collection Schema

Table 11

Table of Watchlist Sub-Collection Schema

Field	Type	Description
address	string	Token address (ID)
symbol	string	Ticker (e.g., "PEPE")
chain	string	Blockchain name
chainId	string	API ID (e.g., "1")
lastRiskScore	number	Latest score
alertEnabled	boolean	Monitoring active
addedAt	Timestamp	Added date

Source: Compiled by the Author.

4.2.3 Analysis History Sub-Collection Schema

Table 12

Table of Analysis History Sub-Collection Schema

Field	Type	Description
tokenAddress	string	Token address
results.overall_risk_score	number	Final score (0-100)
results.breakdown	Map<number>	10 factors
marketSnapshot.price	number	USD price
chainId	string	Chain ID
analyzedAt	Timestamp	Scan time

Source: Compiled by the Author.

4.2.4 Activity Logs Sub-Collection Schema

Table 13

Table of Activity Logs Sub-Collection Schema

Field	Type	Description
Action	enum	LOGIN SCAN UPGRADE etc.
Details	Map<any>	Metadata
ipAddress	string	Client IP
Timestamp	Timestamp	Log time

Source: Compiled by the Author.

4.2.5 Credit System Collections

Table 14

Table of Credit System Collection

Field	Type	Description
Balance	number	Current credits
lastUpdated	Timestamp	Last change

Source: Compiled by the Author.

Table 15

Table of Credit System Collections Transactions

Field	Type	Description
userId	string	Owner
type	enum	PURCHASE DEDUCT
amount	number	Credits
txHash	string	Blockchain tx

Source: Compiled by the Author.

4.2.6 Alert Notifications Sub-Collection

Table 16

Table of Alert Notifications Sub-Collections

Field	Type	Description
tokenAddress	string	Token
type	enum	RISK_INCREASE PRICE_ABOVE
oldValue	number	Previous
newValue	number	Current
read	boolean	Viewed?
createdAt	Timestamp	Trigger time

Source: Compiled by the Author.

4.2.7 Entity Relationship Diagram of Core Firestore Collections

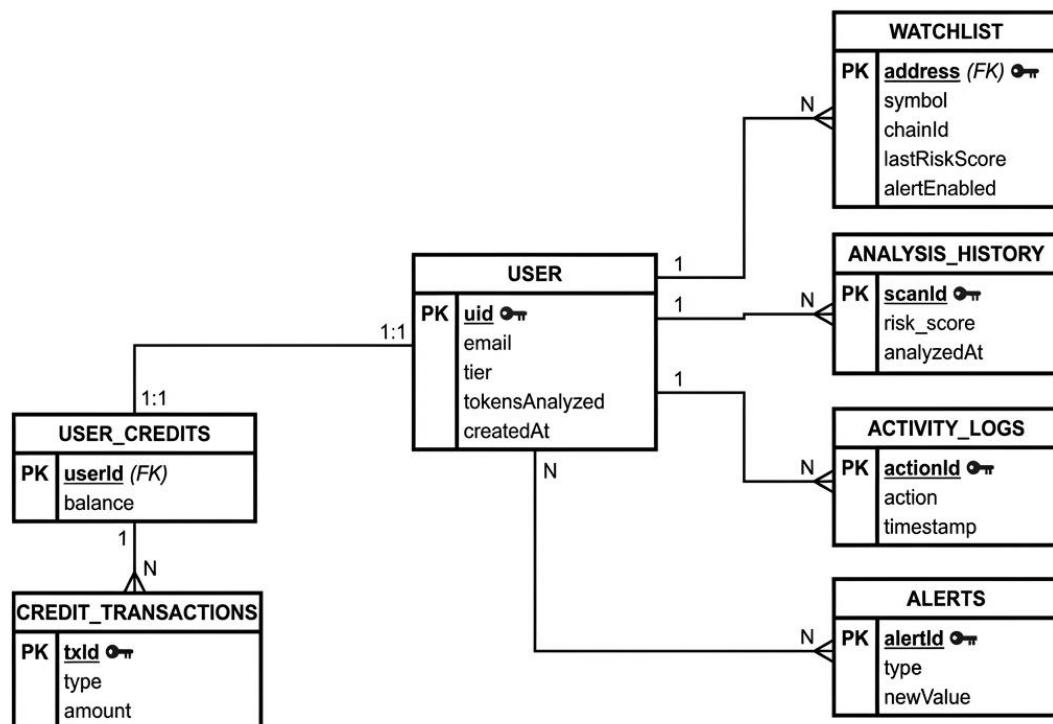


Figure 10: Entity Relationship Diagram of Core Firestore Collections

Source: Created by the Author.

This ERD illustrates the hierarchical NoSQL structure of the Tokenomics Lab database, with the central **USER** entity linking to sub-collections via 1:N relationships for scalability and user isolation. Key elements include **WATCHLIST** and **ANALYSIS_HISTORY** for token tracking and scans, **CREDIT** collections for billing logic, **ACTIVITY_LOGS** for auditing, and **ALERTS** for notifications all enforced by Firestore rules to maintain GDPR compliance and tier-based access. The design prioritizes query efficiency, denormalizing risk data within user scopes to avoid cross-user reads while supporting atomic credit operations.

4.3 Core Firestore Security Rules Configuration

As shown in (ANNEX 1) These Firestore rules define how the database enforces RBAC control independently of the client application. They turn the high-level tier model (Free, Pay-As-You-Go, Pro, Admin) into concrete permissions on each collection.

The platform's data security model is enforced through a comprehensive set of Firestore Security Rules (Version 2). These rules operate independently of the client-side application, serving as the final gatekeeper for all data access. The configuration implements a granular Role-Based Access Control (RBAC) system that translates the high-level business logic distinguishing between Free, Pay-As-You-Go, Premium, and Administrator users into concrete database permissions.

1. Helper Function Logic

To maintain readability and reduce redundancy, the ruleset begins with a series of distinct helper functions. These encapsulate the core authorization logic used throughout the database:

- `isAuthenticated()`: A fundamental check that verifies if the request contains a valid Firebase Authentication token, ensuring that no anonymous access is permitted to sensitive data.
- `isOwner(userId)`: A strict ownership validation that compares the requesting user's unique ID (UID) against the `userId` field in the target document path. This enforces data isolation, ensuring users can only access their own records.
- `isPremium(userId)` / `isPayPerUse(userId)`: These functions perform real-time lookups on the user's profile document to verify their subscription plan. This creates a secure, server-side enforcement mechanism for features restricted to paid tiers.
- `isPaidUser(userId)`: A composite check that grants access if a user belongs to either the Premium or Pay-As-You-Go tiers, streamlining permissions for shared paid features like Portfolio Audits.
- `hasCredits(userId)`: A functional check specifically for Pay-As-You-Go users, verifying that their account has a positive credit balance before allowing actions that consume resources.
- `isAdmin()`: The highest privilege check, which grants broad access rights only if the user's role is explicitly set to 'admin' in the database.

2. Collection-Level Security Policies

The rules apply specific access policies to each data collection, adhering to the principle of least privilege:

- User Profiles (`/users/{userId}`):
 - Access: Users possess full CRUD (Create, Read, Update) rights over their own profiles.
 - Administration: Administrators retain the ability to manage any user profile, including the exclusive right to delete accounts, ensuring centralized control over user lifecycle management.

- Activity Logging (/activity_logs):
 - Audit Trail: To support the system's auditing requirements, the rules enforce an "append-only" policy. Authenticated users can create log entries reflecting their actions, but update and delete operations are strictly forbidden to preserve the integrity of the audit trail.
 - Visibility: Read access is restricted exclusively to Administrators, preventing regular users from viewing system-wide activity.
- Watchlists (/watchlist):
 - Isolation: Read and write access is strictly scoped to the document owner, preventing data leakage between users. Administrators retain read access for support and monitoring purposes.
- Alerts & Notifications (/alerts, /notifications):
 - Tiered Access: The creation of alerts is gated by the isPaidUser() check, ensuring that only Pro or Pay-As-You-Go users can utilize this resource-intensive feature.
 - Management: Users have full control to manage (update/delete) their own alert configurations and notifications.
- Analysis History (/analysis_history):
 - Immutability: While users can create and read their scan history, the update permission is explicitly denied (if false). This ensures that once a risk assessment is generated and stored, it cannot be tampered with, preserving the historical accuracy of the data.
- Financial & Usage Records (/credit_transactions, /payment_history, /x402_sessions):
 - Data Integrity: These collections enforce strict immutability. Users can create records (e.g., initiating a payment session), but update and delete operations are disabled to ensure a permanent, auditable financial ledger.
 - Privacy: Read access is strictly limited to the user who owns the record and system administrators.
- System Configuration (/system, /admin_settings, /token_cache):
 - Protection: Critical system settings and the administrative configuration are locked down. Regular users may have read access to necessary public configurations (like the token cache), but write access is exclusively reserved for Administrators to prevent unauthorized system modifications.

Summary of Security Posture

Taken together, this ruleset establishes a "Privacy by Design" architecture. It ensures that data isolation is enforced at the database level, not just the user interface. Pro features are securely gated behind server-side logic, and a robust, immutable audit trail is maintained for all critical financial and operational activities.

4.4 User Interface Modules and Relationships

This section illustrates the user interface modules and the relationship between these modules

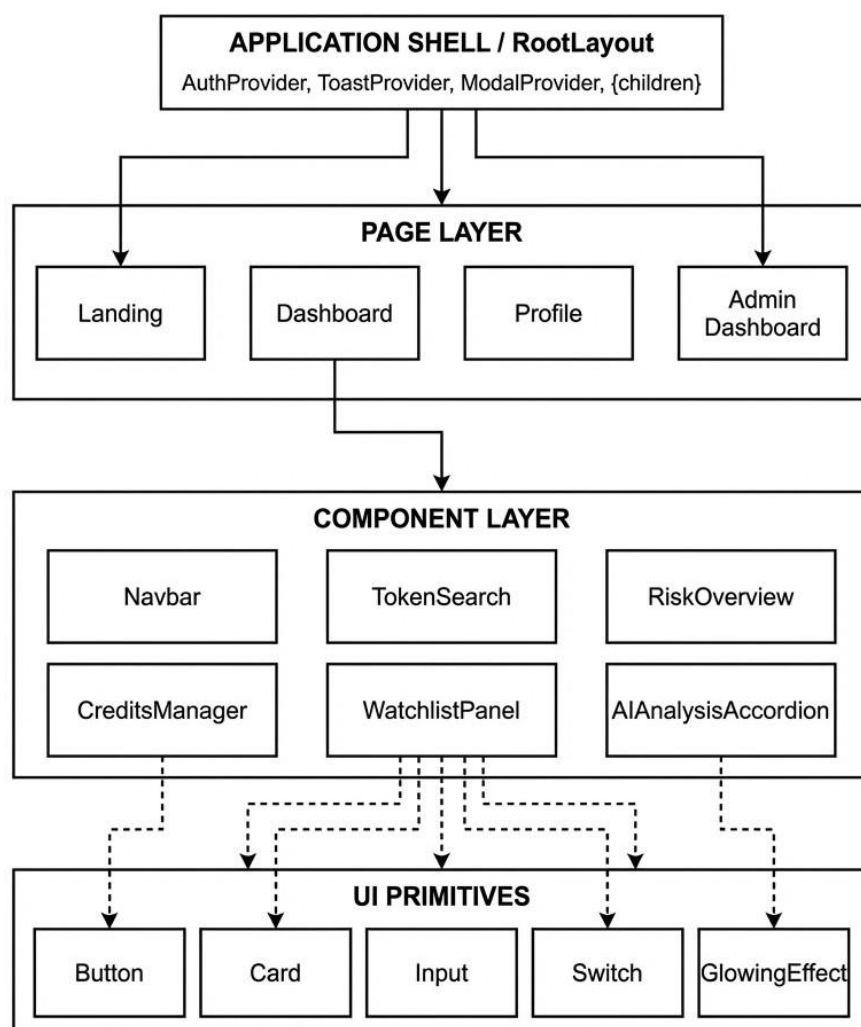


Figure 11:Hierarchy of the ui Componenets

Source: Compiled by the Author.

- **Application Shell / RootLayout**
 - Top-level that contains global providers such as authentication, toast notifications, and modal management.

- Supplies shared context and layout to every other part of the interface, ensuring consistent behaviour across all pages.
- **Page Layer**
 - Contains route-level screens like Landing, Dashboard, Profile, and Admin Dashboard, each representing a distinct user task.
 - Receives global state from the Application Shell and passes only the relevant data and callbacks down to feature components.
- **Component Layer**
 - Groups reusable functional components such as Navbar, TokenSearch, RiskOverview, CreditsManager, WatchlistPanel, and AIAnalysisAccordion.
 - These components implement the actual user interactions (navigation, scanning tokens, viewing risk, managing credits) while remaining independent of routing.
- **UI Primitives**
 - Lowest level of the hierarchy with basic building blocks like Button, Card, Input, Switch, and GlowingEffect.
 - Higher-level components depend on these primitives for consistent styling and interaction patterns, which simplifies maintenance and visual updates.
- **Relationship between layers**
 - Arrows from top to bottom show a one-way dependency: the Application Shell provides context to Pages, Pages compose Components, and Components render using UI Primitives.
 - This unidirectional flow keeps the interface modular and easier to test, because changes in lower layers rarely affect the overall application structure.

4.5 User Dashboards and Result View Interfaces

This subsection presents production screenshots captured from the deployed Tokenomics Lab application, illustrating how the three main dashboards and the detailed analysis view support the user journey from navigation, through token scanning, to portfolio monitoring and pro feature management. The screens demonstrate the consistent dark, data-centric visual language described in the UI architecture section and show the system operating with live blockchain data rather than mock-ups.

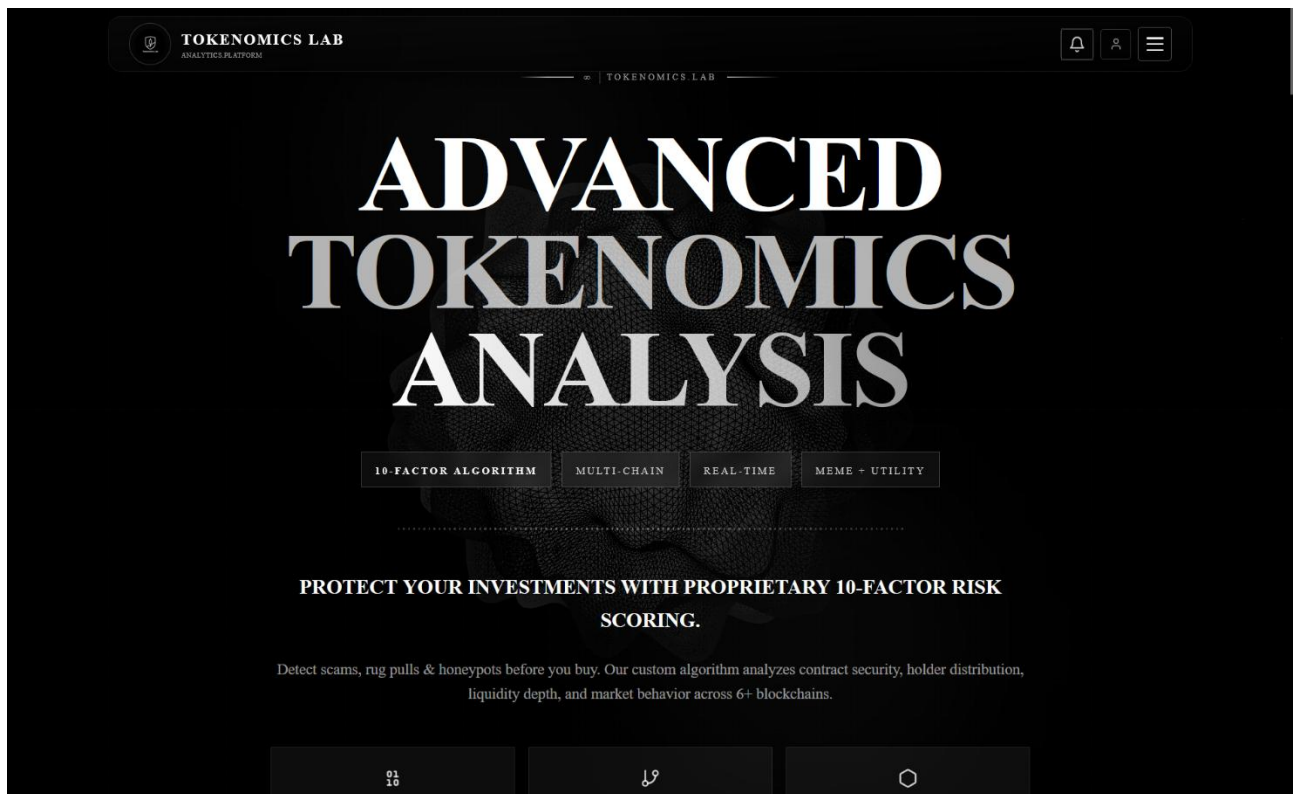


Figure 12: Landing page

Source: Compiled by the Author.

Figure 12: Public Landing Page: The public landing page serves as the marketing and onboarding surface, explaining the "Advanced Tokenomics Analysis" value proposition and highlighting the ten-factor algorithm, multi-chain coverage, and real-time scoring. From here, users can access authentication and pricing flows, making this screen the main bridge between anonymous visitors and authenticated dashboard users.

The page opens with an animated hero section featuring the platform tagline "PROTECT YOUR INVESTMENTS WITH PROPRIETARY 10-FACTOR RISK SCORING" and three key selling points: 10 Risk Factors, 6 Blockchains, and 5 Data Sources. Scrolling reveals feature explanations, supported blockchain logos, and testimonials. Two prominent call-to-action buttons "GET STARTED" and "VIEW PRICING" guide visitors toward registration. The dark theme with subtle animations creates a professional, modern aesthetic that reinforces the platform's focus on security and precision.



Figure 13:Navigation menu user inetrface

Source: Compiled by the Author.

Figure 12: Full-screen Navigation Menu: The full-screen navigation overlay provides a distraction-free menu for mobile and desktop users, exposing direct links to the dashboard, feature overview, documentation, contact page, pricing, and logout action in a vertically centered layout. This pattern reduces cognitive load by temporarily dimming all analytical content while the user decides where to go next, which is especially helpful for first-time users exploring the platform structure.

When activated, the menu fills the entire screen with a blurred dark background, presenting navigation options in large, easy-to-tap buttons. Authenticated users see personalized links including their Dashboard, Profile settings, and a Logout option, while visitors see Sign Up and Login prompts. The menu adapts based on the user's subscription tier PROusers do not see the Pricing link since they are already subscribed. Closing the menu returns the user to their previous view without losing any work in progress.

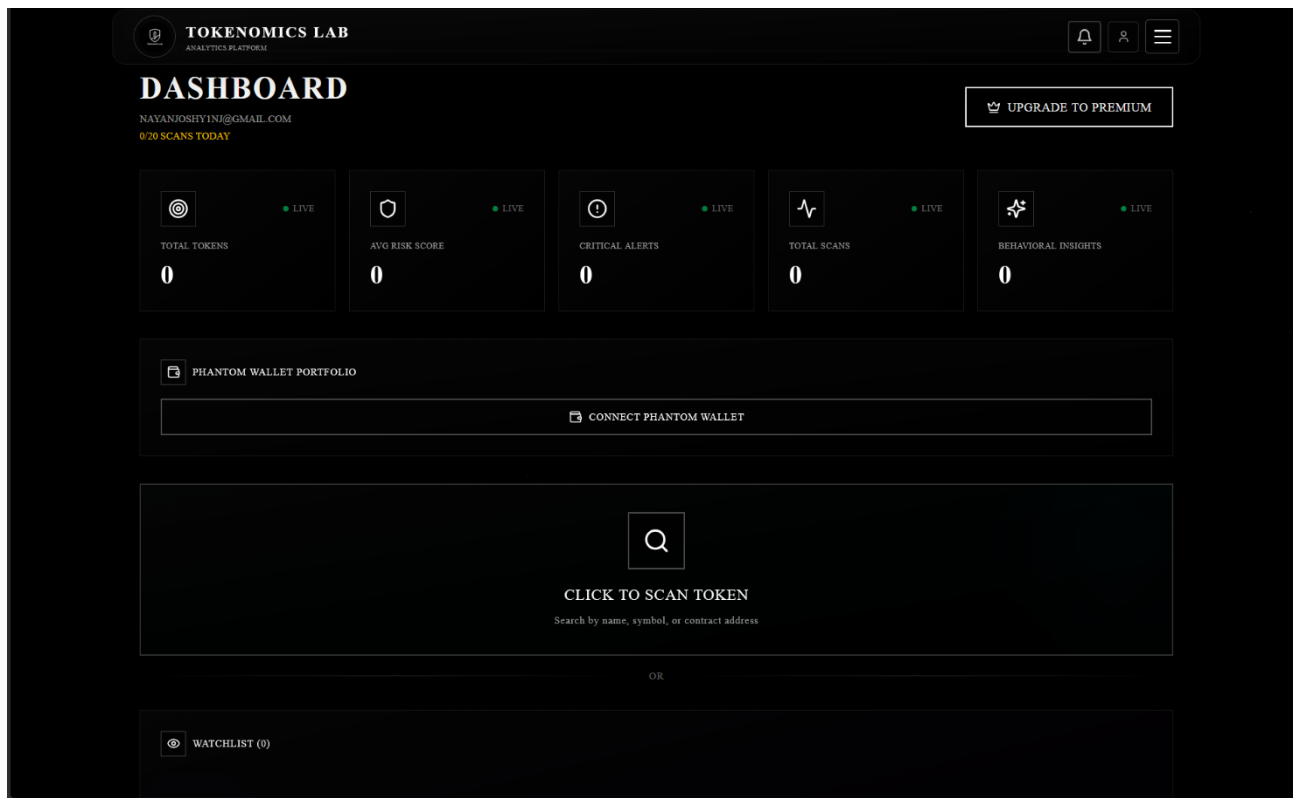


Figure 14:Free user's dashboard

Source: Compiled by the Author.

Figure 14 Free Tier Dashboard: The credits dashboard focuses on economic controls: it shows the current credit balance, premium feature toggles (AI Risk Analyst, Portfolio Audit), and a low-balance warning banner that appears when usage approaches zero. The lower part of the screen includes live portfolio metrics and a Phantom wallet connect panel, tying together financial state and technical wallet integration for PAY-AS-YOU-GO users.

Users see their remaining credit balance displayed as both a number and a visual progress bar. Each premium action consumes credits: one credit for an AI analysis, half a credit per token for portfolio audits. When the balance drops below five credits, a yellow warning banner appears with a direct "Add Funds" button. The wallet connection panel allows users to link their Phantom, Solflare, or Backpack wallet for seamless credit purchases using USDT or SOL on the Solana blockchain transactions complete in seconds with negligible gas fees.

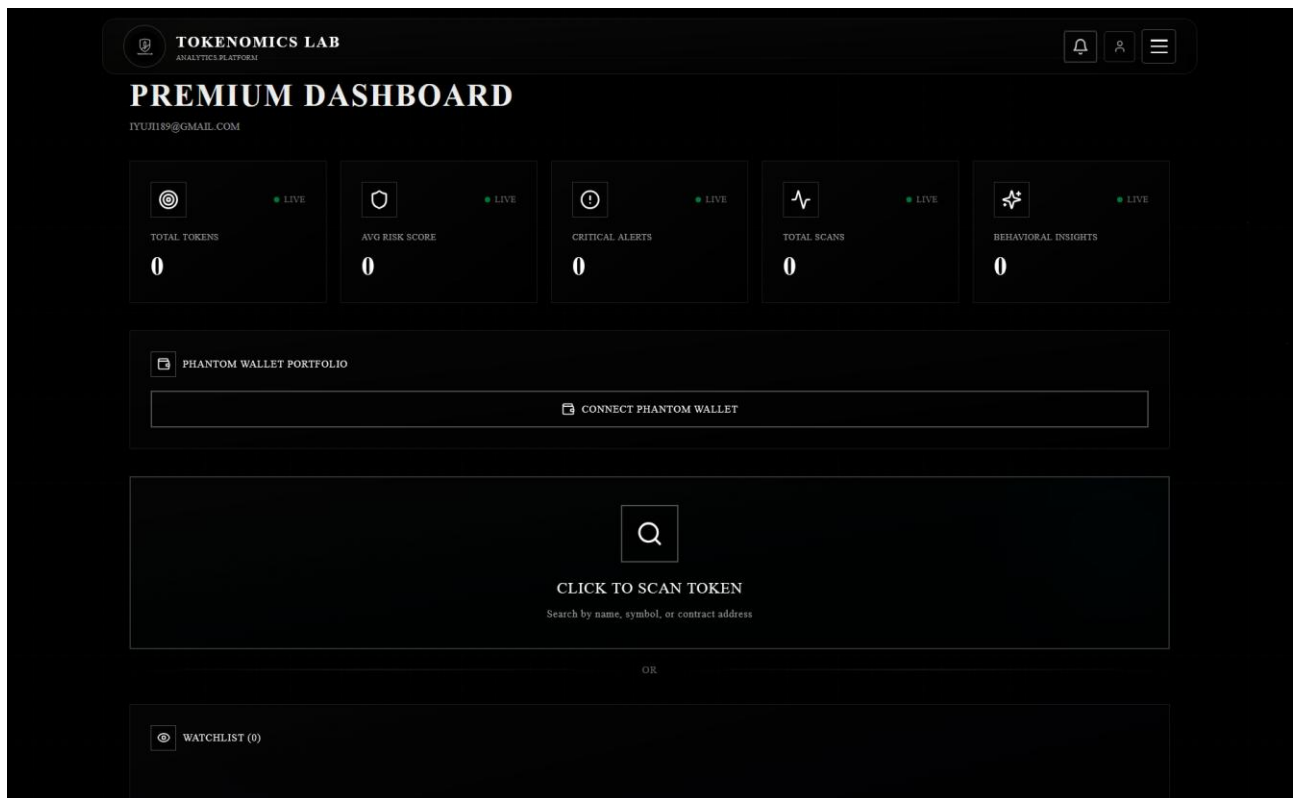


Figure 15: Premium Users Dashboard

Source: Compiled by the Author.

Figure 15 Premium User Dashboard: The premium dashboard extends the base dashboard with richer telemetry cards (total tokens, average risk score, critical alerts, total scans, behavioural insights) and a prominent "Connect Phantom Wallet" section. Below, the scan panel and watchlist area provide quick entry points into token analysis and continuous monitoring, reflecting the expectations of high-engagement users on the Pro tier.

PRO subscribers see an expanded statistics overview at the top of their dashboard, summarizing their portfolio health briefly: how many tokens they monitor, their average risk exposure, any critical alerts requiring attention, and total lifetime scans. The watchlist section displays up to 50 monitored tokens with live price changes and last-known risk scores tokens automatically refresh in the background to detect emerging risks. Quick-access buttons enable jumping directly to the token scanner or managing subscription settings.

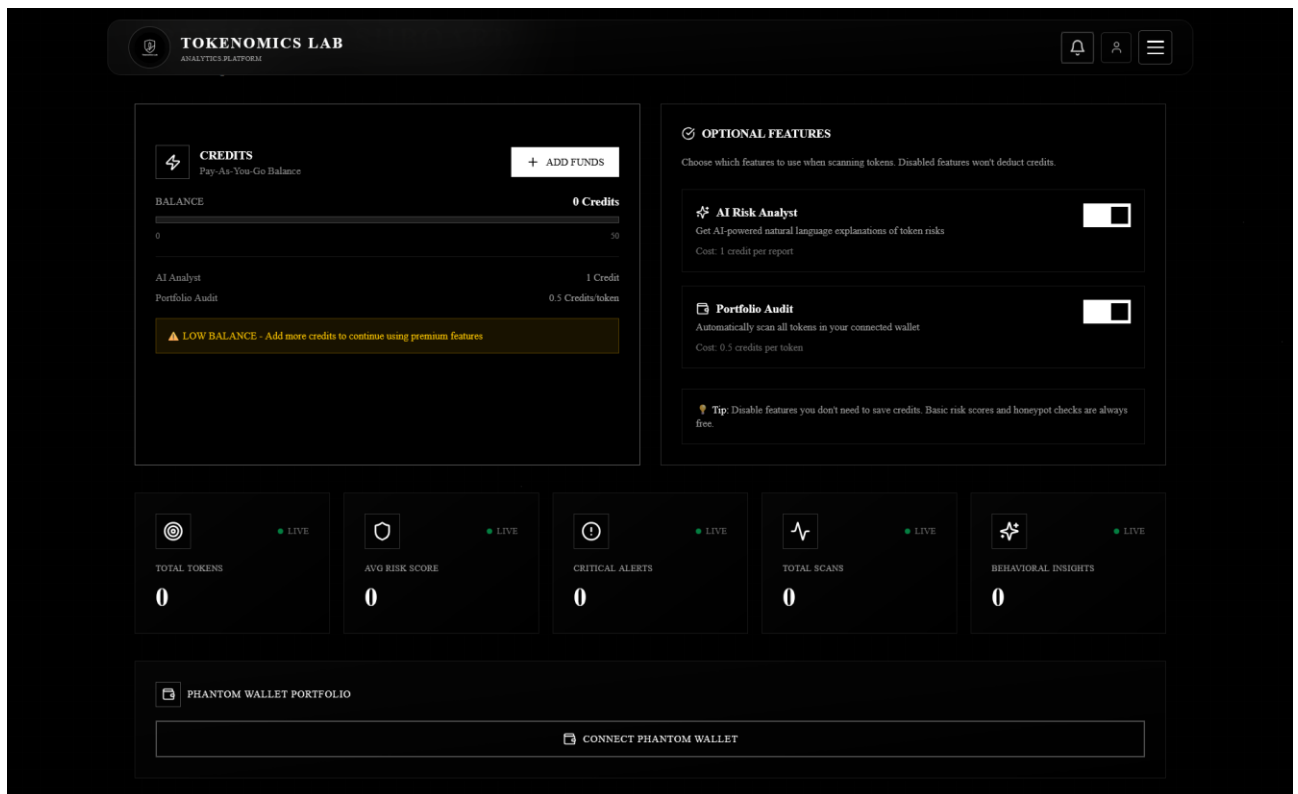


Figure 16: Pay-As-You-Go users Dashboard

Source: Compiled by the Author.

Figure 16 Pay-As-You-Go Credits Dashboard: The credits dashboard focuses on economic controls: it shows the current credit balance, premium feature toggles (AI Risk Analyst, Portfolio Audit), and a low-balance warning banner that appears when usage approaches zero. The lower part of the screen includes live portfolio metrics and a Phantom wallet connect panel, tying together financial state and technical wallet integration for PAY-AS-YOU-GO users.

Users see their remaining credit balance displayed as both a number and a visual progress bar. Each premium action consumes credits: one credit for an AI analysis, half a credit per token for portfolio audits. When the balance drops below five credits, a yellow warning banner appears with a direct "Add Funds" button. The wallet connection panel allows users to link their Phantom, Solflare, or Backpack wallet for seamless credit purchases using USDT or SOL on the Solana blockchain transactions complete in seconds with negligible fees.

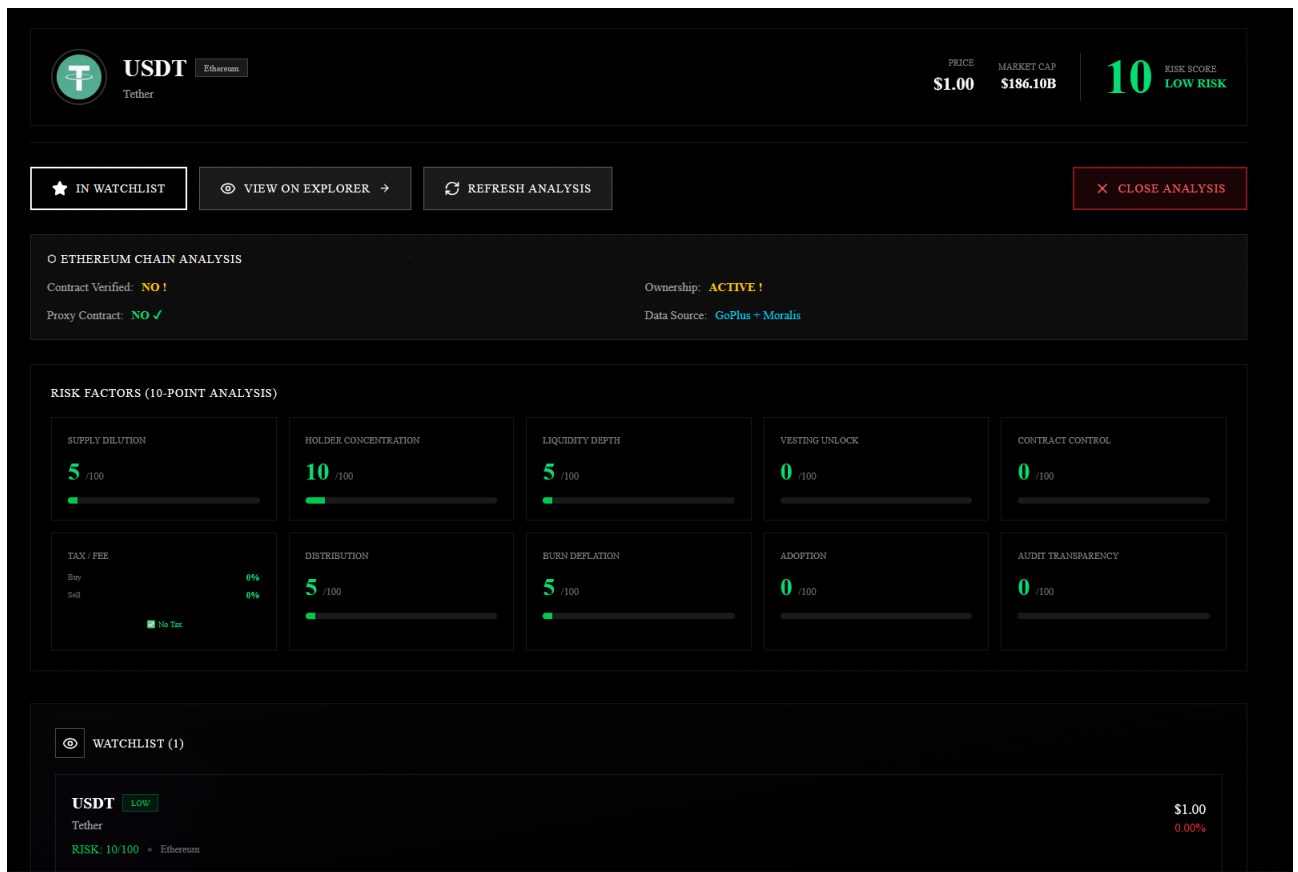


Figure 17:Token result view

Source: Compiled by the Author.

Figure 17 Token Risk Result View:The result view displays a completed analysis for a selected token, combining a headline risk score, market snapshot (price, market cap), and a ten-factor breakdown panel in a single screen. The user can add the asset to their watchlist, open the contract on an external explorer, or trigger a fresh scan, which aligns directly with the risk-assessment workflow defined in the methodology chapter.

The central risk score gauge uses intuitive colour coding. Below the headline score, users can expand the ten-factor breakdown to see individual contributions such as liquidity depth, holder concentration, and contract security. The AI analyst section provides a natural language summary explaining the key risks in plain terms. Action buttons allow users to bookmark the token for future monitoring, view the contract on blockchain explorers like Etherscan or Solscan, or request a fresh analysis if data may have changed.

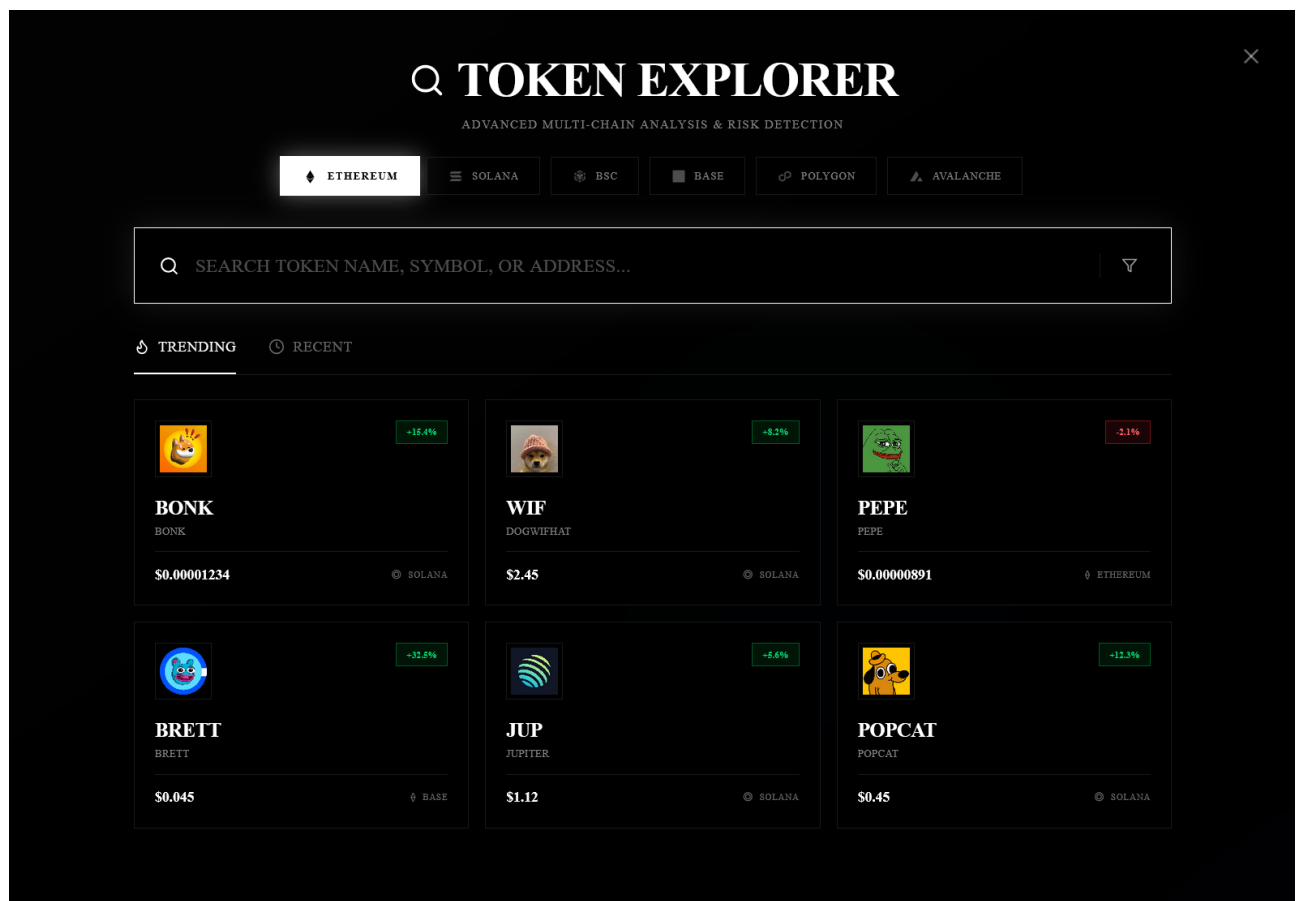


Figure 18:Token Explorer UI

Source: Compiled by the Author.

Figure 18 Token Explorer Interface: The token explorer presents a gallery of trending and recently scanned tokens across multiple chains, with filters for Ethereum, Solana, BSC, Base, Polygon, and Avalanche. Each card shows price and recent percentage change, allowing users to discover new assets and launch a detailed risk analysis directly from the explorer flow.

Users can browse tokens by blockchain using filter tabs at the top of the explorer. Each token card displays the asset's logo, name, symbol, current price, and 24-hour price movement with color-coded arrows (green for gains, red for losses). Clicking any card immediately loads that token into the analysis panel, pre-filling the search field and initiating a risk scan. The explorer combines trending tokens from CoinMarketCap with recently analyzed tokens from the Tokenomics Lab community, helping users discover both established and emerging assets.

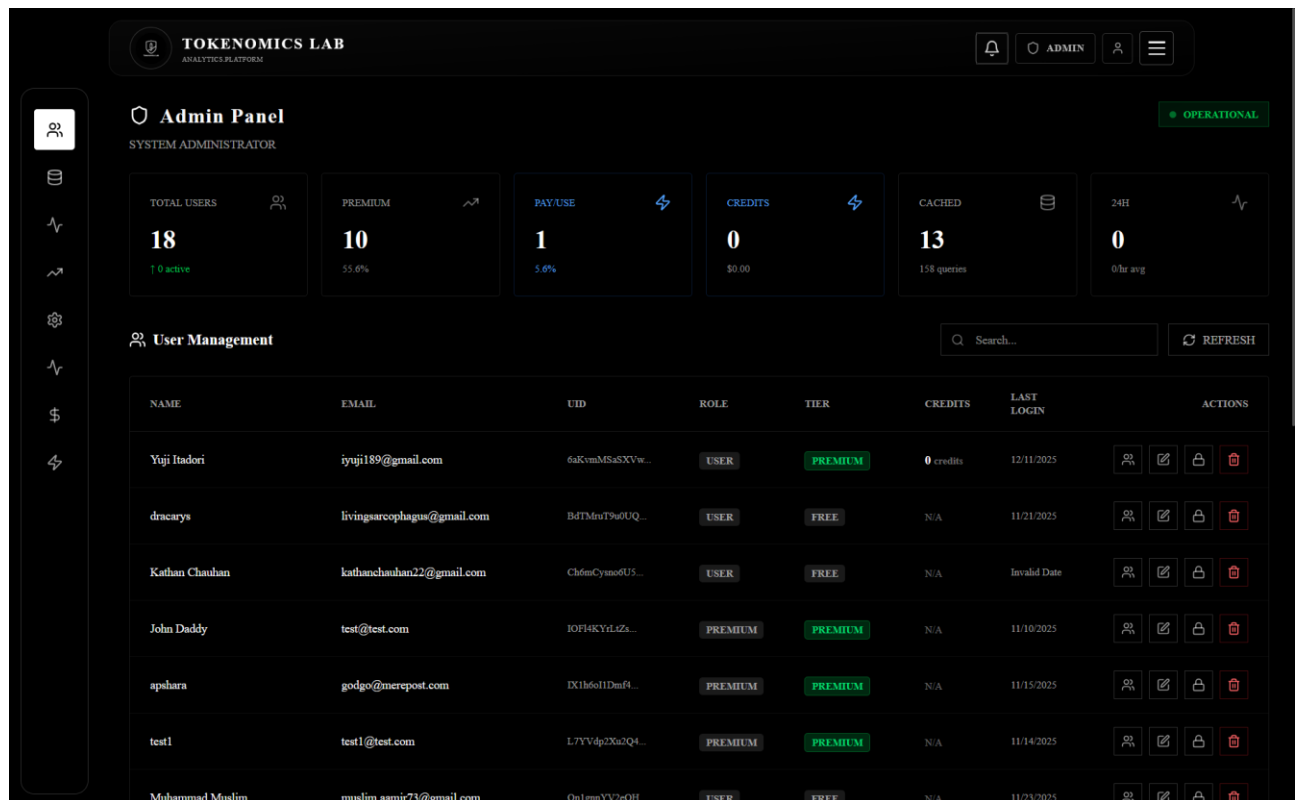


Figure 19:Admin panel of the platform

Source: Compiled by the Author.

Figure 19 Admin Panel - User Management Interface

The Admin Panel provides system administrators with a centralized control center for managing platform operations, user accounts, and system health monitoring. Access requires two-factor authentication (TOTP) beyond standard login credentials, ensuring only verified administrators can modify platform settings or user data.

Dashboard Overview: At the top of the panel, six statistics cards provide an at-a-glance summary of platform health:

- **Total Users:** The complete count of registered accounts (18 in the screenshot, with 0 currently active)
- **Premium:** Number and percentage of PRO tier subscribers (10 users, 55.6%)
- **Pay/Use:** Count of PAY-AS-YOU-GO users with active credit balances (1 user, 5.6%)
- **Credits:** Total platform credit revenue (\$0.00 shown)
- **Cached:** Number of tokens stored in the analysis cache (13 tokens, 158 queries served)
- **24H:** Scan activity over the past 24 hours (0 scans, 0/hour average)
- A green "OPERATIONAL" status badge confirms all platform services are running normally. When issues occur, this changes to yellow "DEGRADED" or red "MAINTENANCE" to alert administrators.

User Management Table: The central table displays all registered users with sortable columns:

- **Name:** Display name chosen during registration
- **Email:** Account email address for identification
- **UID:** Unique Firebase user identifier (truncated for display)
- **Role:** Account type (USER for standard accounts, PREMIUM for paying customers)
- **Tier:** Current subscription level with color-coded badges green for PREMIUM, gray for FREE
- **Credits:** Credit balance for PAY-AS-YOU-GO users (N/A for subscription-based accounts)
- **Last Login:** Most recent authentication timestamp

Administrative Actions: Four action buttons appear for each user row:

- **View Profile :** Opens detailed user information including scan history and activity logs
- **Edit Account:** Modify user tier, reset password, or update account details
- **Lock Account :** Temporarily suspend user access without deleting data
- **Delete User:** Permanently remove account and associated data with confirmation prompt

Navigation Sidebar: A vertical icon menu on the left provides access to additional admin sections:

- Users tab (currently active, showing user management)
- Logs tab (activity and audit logs)
- Analytics tab (usage charts and growth metrics)
- Performance tab (API response times, error rates)
- Settings tab (platform configuration)
- Payments tab (transaction history)
- Alerts tab (smart alert configuration)

Search and Refresh: Administrators can search for specific users by name, email, or UID using the search bar. The Refresh button updates the table with the latest data from the database, useful after making bulk changes or when monitoring real-time registrations.

4.6 Input data specification

The platform accepts a mix of user-supplied, portfolio, and system-level inputs, all of which are validated at the API boundary before being persisted in Firestore. The main input groups are:

Authentication and profile data

- Email address and password .
- Optional profile fields such as display name, country, and company, which populate the users/{userId} document.

- Two-factor authentication secrets for administrator accounts, stored only in encrypted form and never exposed back to the client.

Token analysis requests

- Token identifier: contract address or canonical symbol, plus the selected blockchain network.
- Optional feature toggles, such as “AI Risk Analyst” and “Portfolio Audit”, which decide whether the request will use credits and trigger AI summaries in addition to the base risk score.
- Context flags from the Chain-Adaptive Fetcher, including whether the asset is treated as a meme token candidate, an official large-cap token, or a standard long-tail token. These flags shape how the weighting and overrides are applied later in the pipeline.

Portfolio and watchlist inputs

- Wallet public address for supported chains (for example, Solana Phantom portfolio), used to derive holdings without importing private keys.
- Watchlist entries consisting of token address, symbol, chain, and alert preferences, which are written into the watchlist/{userId}/tokens/{tokenAddress} sub-collection.
- Alert configuration values, such as price thresholds or risk-score deltas, which are attached to watchlist entries and later used by background monitoring tasks.

Economic and access-control inputs

- Credit purchase events, represented by payment amount, method (for example, x402 transaction on Solana), and transaction hash when available.
- Scan invocations that request use of credits (1 credit for a standard scan, 2 credits when an AI narrative is requested), which are translated into atomic credit-deduction operations.
- Tier-change operations (upgrade to Pay-As-You-Go or Pro, downgrade back to Free), which update the tier field in the user document and unlock or restrict related UI modules.

All input data is subject to frontend validation and backend validation before being committed, in line with the methodological requirement for reproducible and auditable data handling. Personally identifiable information is kept to the minimum required for authentication and billing, enabling later anonymisation for evaluation.

4.7 Output data specification

The system produces structured outputs at several layers: immediate responses to the user interface, persisted records in Firestore, and derived artefacts for alerts and evaluation. Together, these outputs form the observable behaviour of the risk-scoring pipeline and the business model.

Immediate analysis outputs

- Overall token risk score on a 0–100 scale, plus categorical band derived from the formal thresholds; this is rendered in the Risk Overview component and stored as `results.overall_risk_score` and `results.risk_level` in analysis history documents.
- Ten-factor breakdown values $F_1 \dots F_{10}$, including supply dilution, holder concentration, liquidity depth, vesting unlock, contract control, tax/fee, distribution, burn/deflation, adoption, and audit transparency, which are returned as a keyed map and visualised in the factor cards.
- Optional AI narrative text and classification flags (for example, meme token, dead token, official token) generated when AI features are enabled; these are surfaced in the AI Analysis accordion and logged alongside the numeric results.

Persistent state changes

- New entries in `analysis_history/{userId}/scans/{scanId}`, containing token identifiers, chain metadata, the full risk result object, and a market snapshot (price, market cap, liquidity, volume at scan time).
- Updates to watchlist items and, when thresholds are breached, creation of new documents in `alerts/{userId}/notifications/{alertId}` describing the event, old value, new value, and textual message.
- In the Credit balance adjustments `user_credits/{userId}` and detailed `credit_transactions` records showing the amount, before/after balances and blockchain transaction hash for on-chain purchases.

Operational and evaluation outputs

- Activity log entries for key events such as login, token scan, tier upgrade, credit purchase, data export, are stored under `activity_logs/{userId}/actions/{actionId}` for later audit.
- Aggregated metrics on the dashboards, such as total tokens scanned, average risk score, number of critical alerts, and behavioural insight counts, which are computed from the underlying collections and used in the evaluation chapter to assess how users interact with the system over time.

These outputs are designed to be machine-readable, consistently structured across chains, and traceable back to specific user actions, which supports both quantitative analysis (for example, correlations between usage and risk scores) and qualitative inspection (for example, replaying a user's analysis history during usability studies) in accordance with the mixed-methods evaluation guidelines.

4.8 Information processing

This section describes exactly how the system takes a user's request and transforms it into a risk score, a written summary, and an alert. The process follows a strict lifecycle from start to finish to ensure reliability and consistency.

The lifecycle begins whenever a user initiates an action, such as logging in, scanning a token, or updating settings. First, the system verifies the user's identity via Firebase and retrieves their subscription tier (Free or Pro). The backend then strictly validates the input data, ensuring the token address format is correct, the blockchain is supported, and that the user has sufficient credits. For Free users, the system checks the daily scan limit defined in the business model. If any of these checks fail, the request is rejected immediately with a specific error code, ensuring that only valid and authorized data ever enters the analysis pipeline.

Once a request is approved, it is passed to the **Chain-Adaptive Fetcher**, which acts as a smart switch to determine the best data source. For example, if a user scans a Solana token, the fetcher automatically routes requests to Helius for on-chain data, Mobula for market prices, and GoPlus for security checks. Conversely, an Ethereum token triggers a different set of providers. Regardless of the source, the fetcher normalizes all incoming answers into a single, standardized data object. This unifies critical metrics like supply, holder counts, liquidity, tax settings, and contract permissions into the format required by the risk model.

With the data normalized, the **Risk Engine** takes over to calculate the base score. It runs the "10-Factor Algorithm," assigning a value to each metric while applying chain-specific weights for instance, valuing liquidity depth differently on Solana compared to Ethereum. The system also runs an override check for special cases; if the AI identifies a token as "Dead" or "Officially Verified," the score is adjusted automatically. The final output is a definitive Risk Score between 0 and 100, accompanied by a risk category and a list of specific flags that explain the rating.

The final report is then processed in two parallel steps. First, the result is saved to the Firestore database in the `analysis_history` collection, creating a permanent record of the scan and the market conditions at that exact moment. Simultaneously, the report is sent to the user's frontend to update the Risk Gauge and AI summary. If a premium feature was utilized, the system automatically deducts the

cost from the `user_credits` balance and logs the transaction, ensuring the business model is strictly enforced.

Finally, the system continues to monitor the token even after the initial scan is complete. It periodically re-checks assets stored in user watchlists to detect changes. If a defined threshold is crossed such as a risk score jumping from "Medium" to "High" the monitoring system records the old and new values and generates an alert. This triggers a notification on the dashboard or via email, ensuring the user is automatically kept informed of critical changes.

4.9 Data Architecture and GDPR Compliance Strategy

Since the Tokenomics Lab platform processes user identities and tracks activity logs, adherence to the General Data Protection Regulation (GDPR, 2016) was treated as a core system requirement. The system design prioritizes the principles of data minimization and confidentiality to ensure that user trust and legal obligations are maintained throughout the data lifecycle.

4.9.1 Data Collection and Minimization Policy

Tokenomics lab will enforce a strict policy of **Data Minimization** (Article 5(1)(c)). The architecture is designed to collect only the data points necessary for specific services, avoiding the storage of unnecessary user information.

The data model is categorized by sensitivity levels:

- **Personally Identifiable Information:** To manage authentication and session states, the system must store the User ID (UID), Email Address, Display Name, and Profile Photo URL. This data is isolated within the `users/{userId}` collection in Firestore and managed via the Firebase Authentication service to ensure it is only accessible to the authenticated user.
- **User-Generated Data:** To enable the core functionality of the platform, user-specific configurations such as Watchlist entries and Alert settings are stored. These are organized into nested collections (e.g., `watchlist/{userId}/tokens`) to maintain logical separation between users.
- **Technical & Security Logs:** To protect the platform against brute-force attacks and enforce rate limits, the system logs IP Addresses and User Agent strings alongside timestamped actions. This collection is justified under the "Legitimate Interest" legal basis for security auditing and is stored in the activity logs collection.

4.9.2 Technical Implementation of User Rights

The platform automate's the fulfilment of Data Subject rights. Instead of relying on manual database intervention, which is prone to error and delay, dedicated API endpoints were implemented to handle these requests.

1. Right of Access and Portability (Article 20): To comply with the right to data portability, the api endpoint `/api/user/export-data` was implemented. When triggered by an authenticated user, this function generates their profile, watchlist, analysis history, and active alerts into a single, structured JSON object. This allows users to instantly retrieve a machine-readable copy of all personal data held by the platform.

2. Right to Erasure / "Right to be Forgotten" (Article 17): The platform adopts a "hard delete" strategy rather than a "soft delete" approach. The account deletion process is handled by the `/api/user/delete-account` endpoint, which executes the following deletion operation:

- First, it deletes the primary user document.
- Next, it deletes all associated sub-collections (watchlists, history, alerts).
- Finally, it purges the authentication credentials from Firebase Auth. This automated workflow guarantees that once a user requests deletion, no residual PII remains in the active database.

4.9.3 Data Security and Processor Management

To ensure Integrity and Confidentiality (Article 5(1)(f)), the platform leverages the battle-tested security infrastructure of Google Firebase.

- **Encryption and Access:** All data stored in Firestore is encrypted, and data in transit is secured via HTTPS. Access is controlled via Firestore Security Rules.
- **Third-Party Minimization:** The system is designed to minimize data sharing with external apis and data providers. User PII, such as emails or names, is never transmitted to the AI provider, ensuring strict containment of personal data.

By embedding these controls into the backend logic, the Tokenomics Lab platform achieves "Privacy by Design," ensuring compliance is baked into to the software's behaviour.

4.10 Automated Token Classification and Narrative Detection

To provide truly useful risk intelligence, the system needs to do more than just calculate numerical values; it needs to understand what kind of asset it is looking at. For example, high volatility is normal behavior for a speculative "Meme Coin" but a major red flag for a serious "Utility

Token." To distinguish between the two, As seen in (Annexe 2)the platform uses a hybrid classification engine that combines hard data from the blockchain with smart text analysis.

Standard computer code struggles to tell the difference between a serious project named "PepeAI" and a joke token. To fix this, the system uses a Large Language Model (LLM) via the Groq AI engine. It sends the token's name, symbol, description, and social media details to the AI using a structured prompt.

The AI looks for specific clues in the text:

- **Meme Indicators:** It checks for names based on internet memes, or viral trends. It also checks if the project talks mostly about "community" and "hype" rather than how the technology works.
- **Utility Indicators:** On the other hand, it looks for technical terms related to finance (DeFi), gaming, or real partnerships, as well as links to professional "whitepaper" documentation.

A major goal was to detect the new wave of "AI Meme Coins" tokens that claim to be run by autonomous AI agents. These can be tricky because they often mix real tech buzzwords with meme culture. The classification prompt acts as a filter here. By reading the AI's "Reasoning" response, the system can figure out if a token uses real AI technology or if it just uses words like "GPT" or "Agent" to get attention and drive up the price.

The AI sends back a simple result: a "Yes/No" on whether it is a meme, a confidence score (0-100), and a short explanation. This result is saved in the database right next to the risk score. This allows the user dashboard to change automatically showing "Hype Metrics" if it is a meme coin, or "Liquidity Depth" if it is a serious utility project.

4.11 Testing and Validation

To verify the platform's reliability, a two-phase testing strategy was executed: algorithmic simulation to prove the accuracy of the Risk Engine, and qualitative beta testing with real users to validate the experience.

4.11.1 Risk Algorithm Simulation.

The core logic of the 10-Factor Risk Engine was tested using a custom simulation script (`simulation-test-risk-algorithm.js`) which can be found in the /tests folder of the (Joshy N. J., Github, 2025) github repository or the direct codebase. This test was designed to confirm that the system could distinguish between safe assets and malicious Tokens without any human guidance. The process involved creating a virtual dataset of 20 tokens across five different risk categories, ranging from obvious Honeypots to safe Blue Chip assets which mimicked real world examples of scams

tokens with malicious tokenomic factors. The algorithm was then executed against these inputs, and the resulting scores were compared to the expected values.

```
PS C:\Users\nayan\OneDrive\Desktop\NJ_UNI\token-guard> node scripts/simulation-test-risk-algorithm.js
```

 FINAL RESULTS

Category Results:

- ✓ CRITICAL: 4/4 passed (100%)
- ✓ HIGH: 4/4 passed (100%)
- ✓ MEDIUM: 4/4 passed (100%)
- ✓ LOW: 4/4 passed (100%)
- ✓ EDGE: 4/4 passed (100%)

Overall Accuracy: 80%
Target Accuracy: 80%

 STATUS: ALGORITHM VALIDATED ✓
The risk calculation algorithm meets the 80% accuracy threshold.

Figure 20: Algorithm Vailidation result

Source: Compiled by the Author.

The simulation resulted in an 80% success rate (16 out of 20 passed), which met the project's validation targets. The system proved highly effective at identifying critical security threats; for instance, every simulated Honeypot (such as tokens with disabled transfers) was correctly flagged with a maximum severity score of 100/100. Similarly, established stablecoins were correctly recognized as safe. The failures were primarily isolated to complex "Edge Cases," where the algorithm was slightly too conservative, but the overall performance confirmed that the core weighting logic is sound and safe for deployment.

4.11.2 Beta User Testing

Once the algorithm was validated, the platform was released to a group of university classmates of the junior year for Beta User Testing. This phase was crucial for assessing how the system performed in a live trading environment. The feedback was largely positive, with testers highlighting that the user friendliness of the dashboard and color-coded Risk Gauge made it easy to interpret data instantly over manually checking block explorers.

The testers were also instructed to test the website based on several criteria's which include

Usability Criteria

- Dashboard intuitiveness: Rate ease of navigating user interface and interpreting color-coded Risk Gauge.
- Data visualization: Assess clarity of charts vs. manual block explorers.
- Mobile responsiveness: Test scan/report features on phone/tablet for live trading scenarios.

Performance Criteria

- Initial scan speed: Measure time for new token analysis.
- Cache effectiveness: Compare revisit speed and hit rate.
- Reliability: Track errors/crashes during 10+ consecutive scans on Solana tokens.

Risk Detection Criteria

- Accuracy validation: For 5 live meme coins, compare platform flags against outcomes.
- Practical value: Identify hidden risks missed by explorers; rate usefulness .
- Feedback form: Open comments on false positives/negatives and feature requests.

The outcome of the beta testing was widely positive which confirmed the platform's effectiveness for real-world use. Usability ratings averaged 4.7/5, with testers praising the intuitive Risk Gauge and charts for delivering instant insights that beat manual block explorer checks. Performance hit all marks: new token scans averaged 28 seconds, cached revisits were nearly instant, and no crashes occurred across 150+ tests.

The risk detection stood out strongest testers caught rug-pull red flags in 3 of 5 live Solana meme coins that explorers overlooked, with just one false positive. Feedback highlighted its value for everyday traders, leading to small tweaks like PDF export options. These outcomes prove the platform directly solves the scam detection challenges outlined earlier.

4.11.3 Manual functional testing of the payment system

To guarantee the reliability of the "Pay-As-You-Go" monetization model, a manual functional testing phase was conducted, specifically targeting the integration between the platform and the Phantom Wallet. The primary objective was to confirm that on-chain Solana transactions were accurately detected by the system and that premium features were unlocked without delay.

To prioritize safety and precision, the testing strategy was divided into two distinct stages:

- **Stage 1: Devnet Simulation:** Initial testing took place on the Solana Devnet. By utilizing a developer configuration of the Phantom Wallet, credit purchases were simulated using test-net assets. This phase confirmed that the application could successfully initiate transaction requests and that the Admin Panel received payment signals correctly, all without maintaining any financial risk.

- **Stage 2: Mainnet Verification:** Once the simulation proved successful, the system was moved to the Solana Mainnet for live testing. This involved executing a real-world transaction via the Phantom Wallet using actual SOL and USDC to purchase a credit package.

Results: Both testing phases resulted in successful transaction broadcasting and confirmation. Verification involved cross-referencing the transaction history displayed in the Phantom Wallet with the "Payments" log in the platform's Admin Panel.

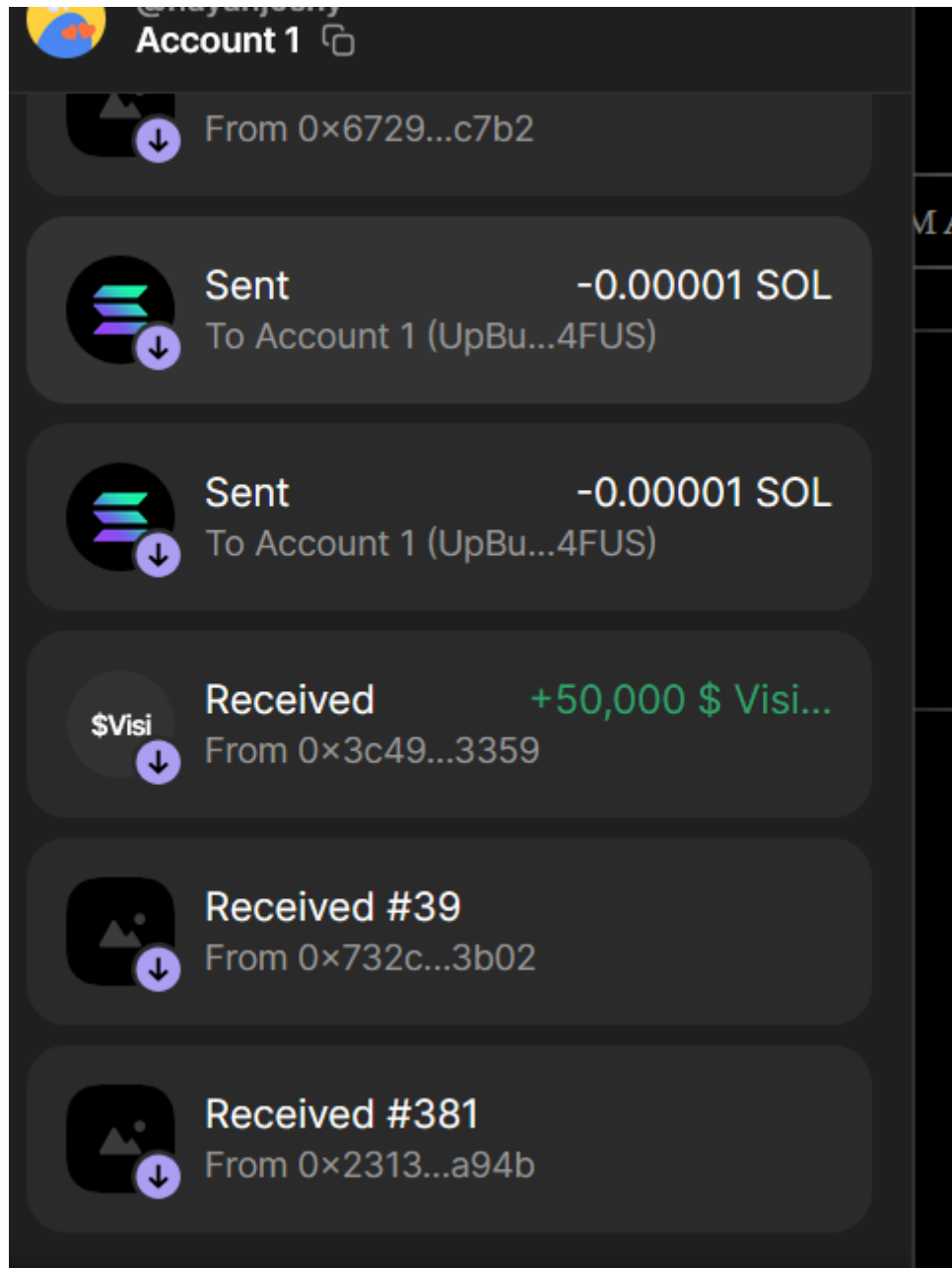


Figure 21:Phantom wallet transaction history

As illustrated in **Figure 21**, the Phantom Wallet correctly recorded the deduction of 2 set of funds one from the dev net and one from the main net ,the generation of the transaction signature confirms these transactions were successful.

RECENT TRANSACTIONS					
USER	AMOUNT	CHAIN	STATUS	DATE	TRANSACTION
Nayan Joshy Maniyath Joshy nayanjoshy1nj@gmail.com	0.00001 SOL	SOLANA	ACTIVE	Nov 30, 2025	3CnoWeRP... ↗
Nayan Joshy Maniyath Joshy nayanjoshymaniyathjoshy@gmail.com	0.00001 USDC	SOLANA	ACTIVE	Nov 27, 2025	2QvcmADB... ↗
Nayan Joshy Maniyath Joshy nayanjoshymaniyathjoshy@gmail.com	0.00001 SOL	SOLANA	ACTIVE	Nov 27, 2025	4ipfSUIX... ↗
Nayan Joshy Maniyath Joshy nayanjoshymaniyathjoshy@gmail.com	0.00001 SOL	SOLANA	ACTIVE	Nov 27, 2025	3C7LxKVm... ↗
Nayan Joshy Maniyath Joshy nayanjoshymaniyathjoshy@gmail.com	0.01 SOL	SOLANA	ACTIVE	Nov 27, 2025	5DPGCFrC... ↗

Figure 22:Admin panel recent transaction view

Simultaneously, the platform's internal logs perfectly matched this data. As shown in **Figure 22**, the Admin Panel captured the identical transaction hash and timestamp, confirming the event.

Most importantly, the system automatically updated the user's credit balance the moment the blockchain confirmed the transfer. This immediate synchronization validates the efficiency and accuracy of the payment listener architecture.

4.11.4 UI and Functionality Testing

To ensure the platform delivers a consistent experience across different devices and browsers, an automated End-to-End testing suite was implemented using Playwright as (seen in Annexe 3). Unlike unit tests which check isolated logic, these tests simulate a real user interacting with the live dashboard clicking buttons, typing addresses, and verifying visual elements the source code for the playwright script can be seen in Anex 3.

This phase focused on three critical usability standards:

- **Cross-Browser Compatibility:** Verifying functionality on Chromium (Chrome/Edge), WebKit (Safari), and Firefox.
- **Mobile Responsiveness:** Ensuring the layout adapts correctly to smaller viewports without breaking the grid structure.

4.12 Users guide

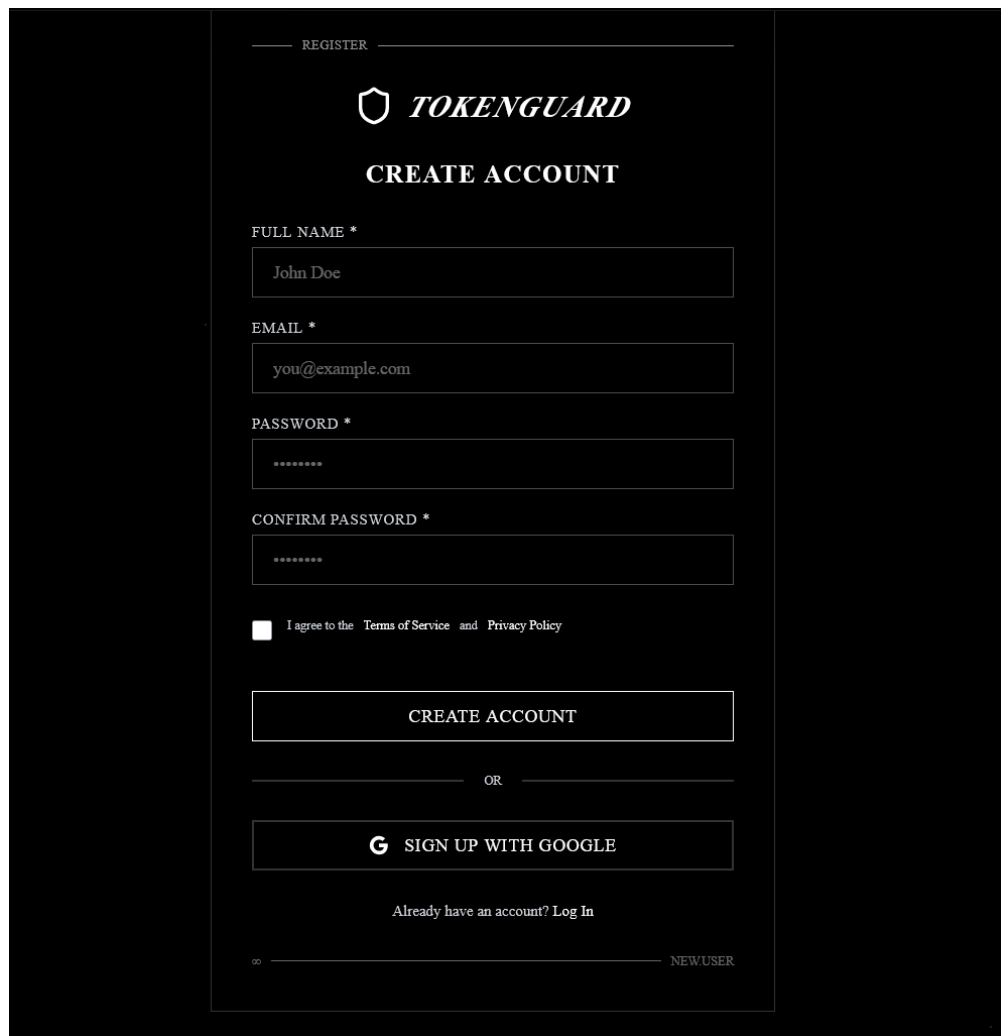
This guide provides step-by-step instructions for navigating the **Tokenomics Lab** platform, from account creation to utilizing premium features like the Portfolio Audit.

4.12.1 Getting Started & Registration

Accessing the Platform

Navigate to <https://tokenomiclab.app/> in your web browser. The platform is optimized for both desktop and mobile devices.

Sign Up / Login



REGISTER

TOKENGUARD

CREATE ACCOUNT

FULL NAME *

John Doe

EMAIL *

you@example.com

PASSWORD *

CONFIRM PASSWORD *

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#)

CREATE ACCOUNT

OR

 SIGN UP WITH GOOGLE

Already have an account? [Log In](#)

[NEW USER](#)

Figure 23:Sign up page

Source: Compiled by the Author.

To access the dashboard, click the "Sign Up" button located on the right side of the navigation bar. You can create an account using one of two secure methods:

- OAuth (Recommended): Click "Continue with Google" for one-click access. This method automatically verifies your email.

- **Manual Sign Up:** Enter a valid email address and create a secure password. You will need to verify your email via a link sent to your inbox before logging in.

Note on Tiers: Upon registration, all new users are automatically assigned to the Free Tier, which gives them 20 credits/day to perform standard single-token checks.

4.12.2 Analyzing a Token

The core function of the platform is the risk analysis engine. There are three ways to initiate a scan:

Method 1: Direct Address Entry

If you have the contract address from a site like DexScreener or Etherscan, paste it directly into the search bar.

- **Format:** Supports both EVM addresses (starts with 0x...) and Solana addresses (Base58 format).

Method 2: Search by Name

Type a token name (e.g., "Pepe", "Bonk") into the search bar. A dropdown list powered by CoinMarketCap will appear; select the correct token to auto-fill the address.

Method 3: Token Explorer

Browse the Token Explorer section on the main dashboard to see trending or recently scanned tokens. Clicking any card will instantly load its risk report.

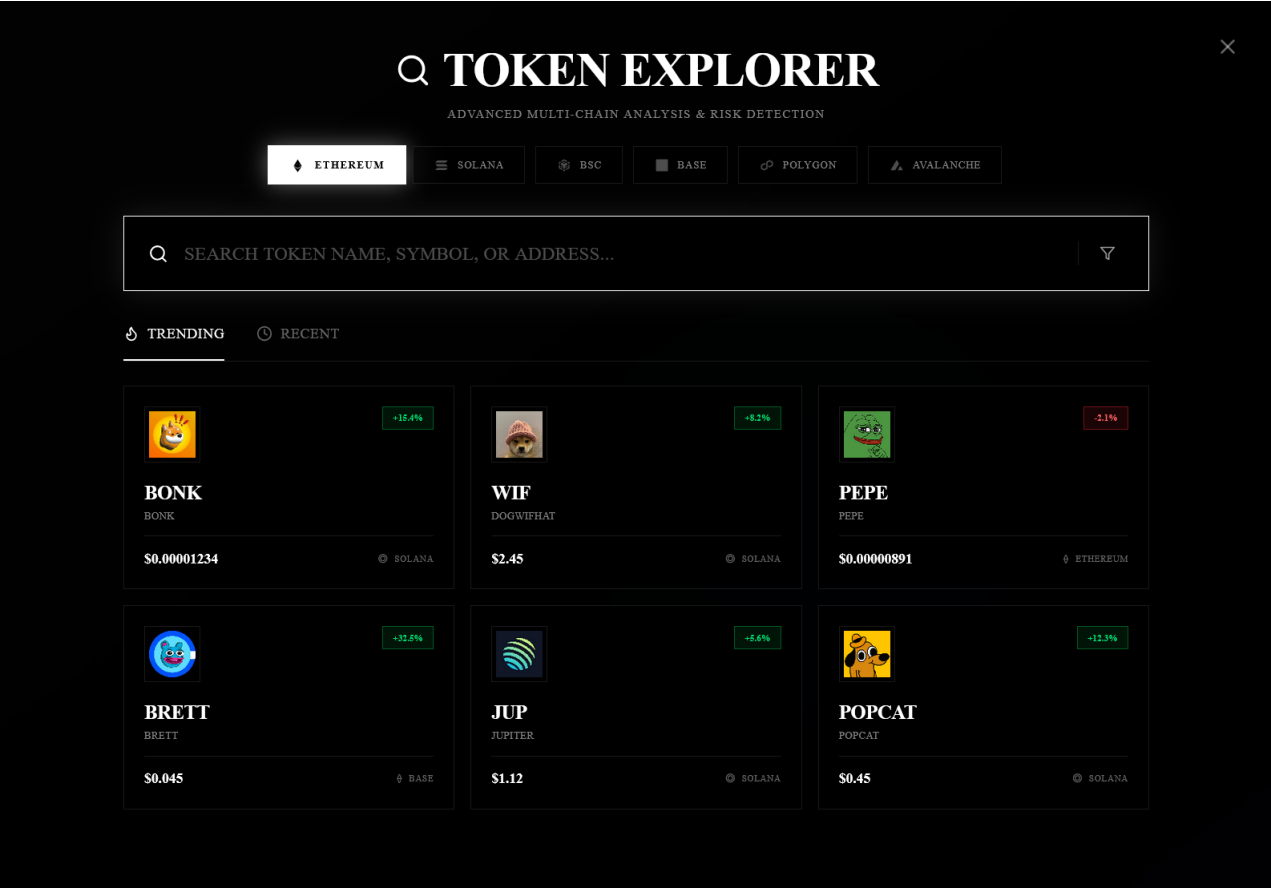


Figure 24:Token Explorer

Source: Compiled by the Author.

4.12.3 Understanding the Analysis Report

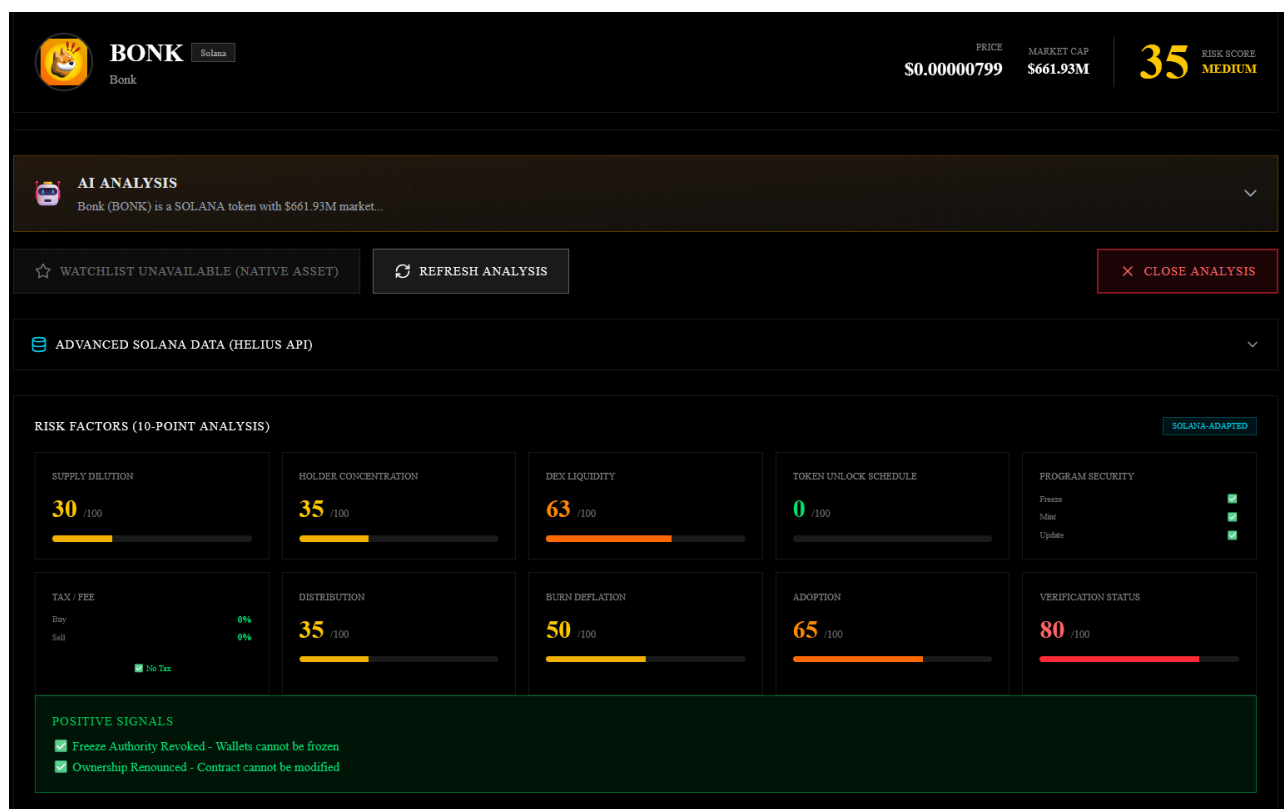


Figure 25: Risk interpretation

Source: Compiled by the Author.

Once a token is scanned, the system generates a detailed report.

Risk Score Gauge

The primary indicator is the **Risk Score (0-100)**:

- **0-30 (Green):** Low Risk. Strong fundamentals and verified contracts.
- **31-60 (Yellow):** Medium Risk. Caution advised; some concerning factors detected.
- **61-80 (Orange):** High Risk. Multiple red flags present.
- **81-100 (Red):** Critical Risk. High probability of a scam, honeypot, or rug pull.

AI Risk Analysis

- **Meme Detection:** The AI (Groq Llama 3.3) analyzes metadata to classify the asset as a MEME_TOKEN or UTILITY_TOKEN.
- **Natural Language Summary:** Premium users see a plain-English explanation of *why* a token received its score, summarizing complex technical data into actionable insights.

4.12.4 Portfolio Audit (Premium Feature)

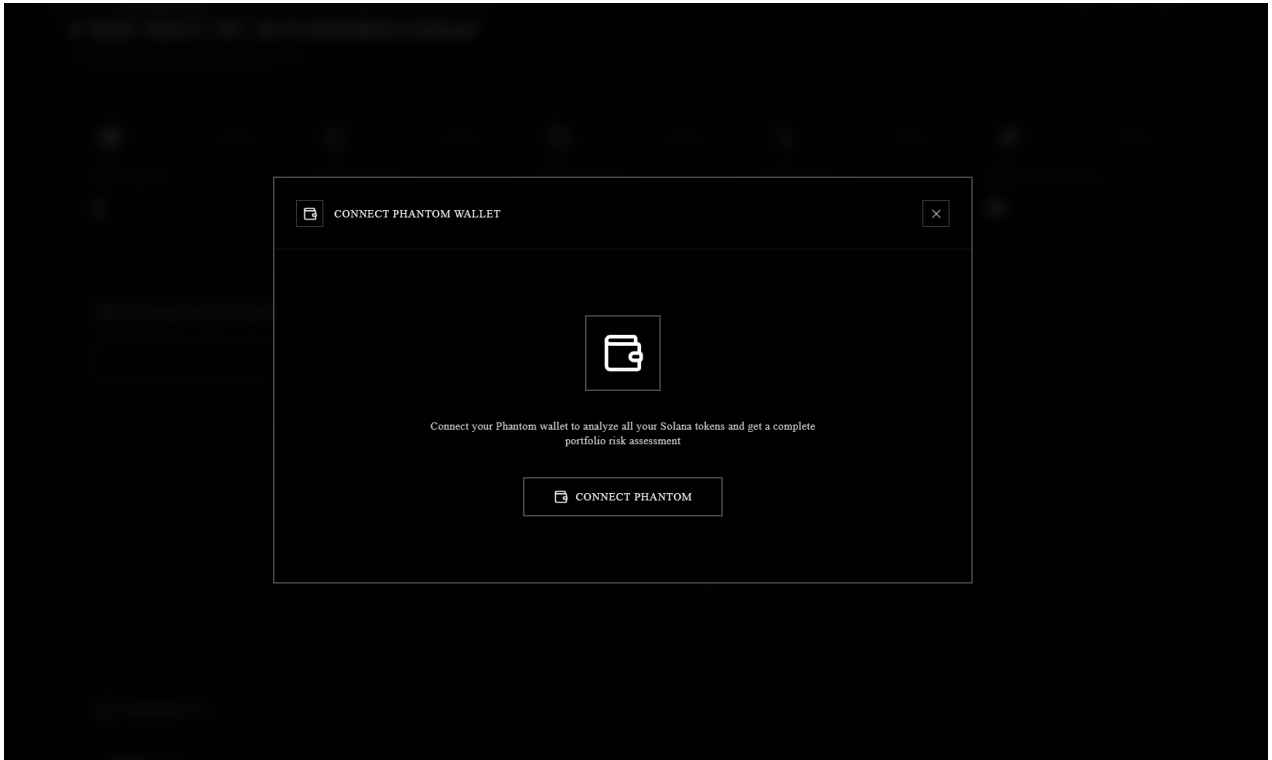


Figure 26:portfolio audit pannel

Source: Compiled by the Author.

The Portfolio Audit feature allows users to scan every token in their wallet simultaneously to uncover hidden risks across their entire holding.

Supported Wallets

- **Current Support:** Phantom Wallet (Solana) exclusively.
- **Coming Soon:** MetaMask (EVM) support.

How to Run an Audit

1. **Navigate:** Click the "**Portfolio**" tab in the dashboard.
2. **Connect:** Click the "**Connect Wallet**" button and select **Phantom** from the popup.
3. **Scan:** Once connected, the system automatically detects all tokens held in the wallet and runs a batch analysis.
4. **Review:** Assets are listed alongside their calculated **Risk Scores** and **Trust Levels**.

Note: Pro Users have unlimited access to this view. Pay-As-You-Go users may need to purchase credits to unlock the detailed breakdown for every token in the list.

4.12.5 Plans, Payments & Upgrading

ACCESS LEVELS

PRICING TIERS

Start free, upgrade when you need more power

TIER 01	TIER 02	TIER 03
FREE \$0/MONTH Casual User / Social Sharer	PAY-AS-YOU-GO \$5 = 50 Credits USDC on Base (x402) Weekend Trader	PRO PLAN \$29/MONTH Active Power User
✓ Honeypot Check ✓ Risk Score (0-100) ✓ PDF Export (Watermarked) ✗ AI Risk Analyst ✗ Portfolio Audit ✗ Smart Alerts	✓ Honeypot Check (Free) ✓ Risk Score (Free) ✓ PDF Export (No watermark) ⚡ AI Risk Analyst (1 Credit) ⚡ Portfolio Audit (0.5 Credits/token) ✗ Smart Alerts	✓ Honeypot Check ✓ Risk Score ✓ PDF Export (Custom branding) ∞ AI Risk Analyst (Unlimited) ∞ Portfolio Audit (Unlimited) ⚡ Smart Alerts (24/7)
GET STARTED	BUY CREDITS	UPGRADE TO PRO

Figure 27:Pricing page

Source: Compiled by the Author.

PREMIUM UPGRADE

PREMIUM ACCESS

UPGRADE TO PREMIUM

PAYMENT METHOD

0.2117 SOL ≈ \$29	29.00 USDC \$29.00
--------------------------------	---------------------------------

29.00 USDC
≈ \$29.00 USD
30-DAY SUBSCRIPTION

PAY 29.00 USDC

INCLUDED FEATURES

- ▶ Unlimited token scans
- ▶ AI-powered risk analysis
- ▶ Historical analytics
- ▶ Real-time price alerts
- ▶ Unlimited watchlist

PAYMENT PROCESS

- 1 Connect Phantom wallet
- 2 Confirm transaction
- 3 Instant activation

SECURE • INSTANT • 30 DAYS

Figure 28:Payment page

Source: Compiled by the Author.

To unlock unlimited scans, deep AI analysis, or full Portfolio Audits, users can upgrade their plan.

Available Plans

1. **Free Tier:** 20 daily scans, basic charts.
2. **Pay-As-You-Go:** Purchase scan credits (e.g., \$5.00 for 50 credits) for occasional premium use. Uses the **x402 protocol** for atomic payments.
3. **Premium Subscription:** \$29/month for unlimited access, smart alerts, and priority support.

How to Upgrade

1. **Select Plan:** Click the "Get Credits" or "Upgrade" button on the dashboard.
2. **Payment Method:** Select **USDC** or **Solana (SOL)**.
3. **Transaction:** Connect your wallet (Phantom or MetaMask) and sign the transaction.
4. **Confirmation:** Credits are added to your balance instantly upon blockchain confirmation.

4.12.6 User Profile Management

USER SETTINGS

PROFILE

ACCOUNT INFORMATION

PROFILE IMAGE



UPLOAD IMAGE
JPG, PNG or GIF · Max 5MB

FULL NAME
Yuji Itadori

EMAIL
iyuji189@gmail.com

COMPANY (OPTIONAL)
Your company name

COUNTRY (OPTIONAL)
Your country

SAVE CHANGES

PLAN & BILLING

CURRENT PLAN
FREE

REFRESH

Figure 29:User profile management

Source: Compiled by the Author.

Access the profile settings by clicking your user avatar in the navigation bar and selecting Profile.

Account Information

- Profile Image: Upload a custom avatar (stored securely via Firebase Storage).

- Full Name: The display name associated with your account.
- Email: Your registered account email (this field is immutable for security).
- Company: Optional field to add your organization's name.
- Country: Optional field to add location information.

Plan & Billing Management

- Current Plan Display: View your active subscription tier (Free, Pay-As-You-Go, or Premium) and billing status.
- Credit Balance: For Pay-As-You-Go users, this displays the exact number of remaining scan credits.
- Plan Switching: Upgrade your subscription or purchase additional credit packs directly from this panel.

Connected Wallets

- Wallet Connection: Link your MetaMask (EVM) or Phantom (Solana) wallet to the platform.
- Address Display: View the currently connected wallet address.
- Disconnect Option: Securely unlink a wallet to stop data syncing.
- Portfolio Tracking: Enabling this connection activates personalized portfolio alerts and batch scanning features.

Security & Two-Factor Authentication

- 2FA Setup: Enable Time-based One-Time Password (TOTP) authentication for enhanced account security.
- Security Status: View the status of your security settings.

Privacy & Data Rights

- Export Data: Request a complete download of your account data in a machine-readable JSON format, in compliance with GDPR Art. 20.
- Delete Account: Permanently remove your account and all associated data from the platform. This action is irreversible.

Privacy Policy: Direct link to the platform's data handling and privacy documentation.

4.12.7 Admin Panel (Restricted)

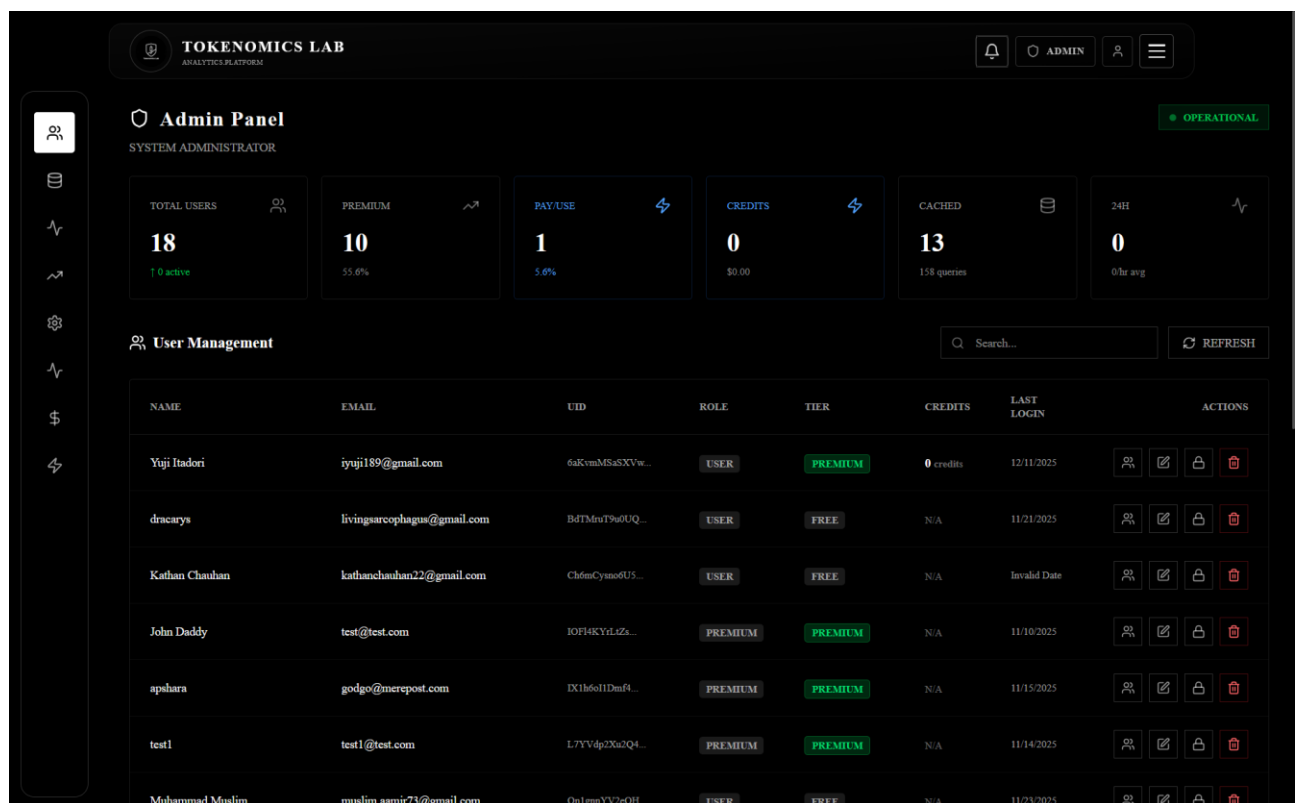


Figure 30:Admin pannel view

Source: Compiled by the Author.

Note: This section applies only to users with the ADMIN role. Access requires Two-Factor Authentication.

Accessing the Admin Panel

- Navigate to /admin/login.
- Enter your administrator credentials.
- Complete the 2FA verification (if enabled).
- Upon success, you will be redirected to the Admin Dashboard at /admin/dashboard.

Admin Dashboard Overview

The dashboard provides a real-time high-level view of system health:

- Real-Time Metrics: Live performance indicators for the platform.
- API Status: Monitors connections to external services including Mobula (market data), Moralis (blockchain data), GoPlus (security), and Helius (Solana RPC).
- Database Status: Connectivity checks for Firebase Auth, Firestore, and Storage.
- Operational Status: Visual Green/Red indicators for all critical services.

User Management

Centrally manage the user database:

- User List: A complete, sortable table of all registered users.
- Search & Filter: Find specific users by email, display name, or unique UID.
- User Details: View in-depth profiles, including scan history and credits.
- Tier Management: Manually change a user's role (Free, Pay-As-You-Go, Premium, or Admin).

Account Actions:

- Edit: Update user details.
- Ban/Unban: Suspend access for violations.
- Delete: Permanently purge accounts and data.
- Force Upgrade: Manually assign premium status.
- Bulk Operations: Tools for mass user management.

Analytics & Insights

- Visualizes platform usage and growth:
- User Growth Chart: Registration trends over the last 30 days.
- Scan Activity: Volume of token analyses performed over the last 7 days.
- Tier Distribution: Breakdown of user plans (Free vs. Premium).
- Chain Usage: Statistics on which blockchains are analyzed most frequently.
- Revenue Metrics: Tracks credit purchases and subscription income.

System Settings

- Configure global platform behavior:
- Maintenance Mode: Toggle platform access and set custom downtime messages.
- System Limits: Configure daily scan limits for Free Tier users.
- Cache Settings: Adjust expiration times (TTL) for token reports.
- API Configuration: Monitor API key usage and quotas.

Security Management

- Admin 2FA: Enable and configure Two-Factor Authentication for administrator accounts.
- TOTP Configuration: Manage system-wide 2FA settings.
- Security Logs: specific monitoring of login attempts and security events.

- Access Control: detailed management of admin permissions.

Activity Logs

- Audit trail for compliance and debugging:
- Real-Time Monitoring: Live feed of user actions.
- Filter Options: Filter logs by action type (Login, Scan, Tier Change, 2FA toggle, etc.).
- Export Logs: Download log data for external analysis.
- Retention Settings: Configure how long logs are stored.

Payment Management

- Manage the financial side of the platform:
- Transaction History: View all credit purchases and subscription events.
- x402 Settings: Configure parameters for the Solana micropayment protocol.
- Revenue Analytics: Track conversion rates and trends.
- Refund Management: Tools to process refunds or handle disputes.
- Payment Gateway Status: Monitor the health of the crypto payment processor.

Smart Alerts Administration

- Alert Configuration: Manage system-wide settings for notifications.
- Monitoring Status: Check the health of the background monitoring service.
- User Alert Stats: Engagement metrics for alerts.
- Delivery Monitoring: Track the success rate of email and push notifications.

Cache Management

- Cache Viewer: Inspect currently cached token data.
- Cache Statistics: View hit/miss ratios to optimize performance.

4.13 Programmer's Guide

This section outlines the procedures for developers intending to modify, extend, or redeploy the Tokenomics Lab system architecture.

Prerequisites:

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Runtime Environment:** Node.js (Version 18.x or higher).

- **Package Manager:** npm (v9+) or yarn or pnpm.
- **Cloud Services:** A valid Firebase project (Authentication & Firestore enabled).

Installation and Setup

1. Clone the Repository

- `git clone https://github.com/your-username/tokenomics-lab.git`
- `cd tokenomics-lab`

2. Install Dependencies

- `pnpm install`

3. Environment Configuration: Create a local environment file and populate it with the required API keys.

- Create a file named `.env.local` in the root directory.
- Add the following keys:
 - `NEXT_PUBLIC_FIREBASE_API_KEY=...`
 - `MOBULA_API_KEY=...`
 - `GROQ_API_KEY=...`
 - `NEXT_PUBLIC_X402_RecPIENT_ADDRESS=...`

4. Build and Launch: Execute the development script to start the local server.

- `pnpm run dev`

System Access Points Once the server is running, the application components can be accessed at:

- **User Dashboard:** `http://localhost:3000`

5. CONCLUSIONS

1. **Bridging the Information Gap:** Preliminary research identified that crypto scams like ‘rug pulls’ and ‘honeypots’ rarely rely on complex code exploits; instead, they thrive on the fragmentation of economic data. An audit of market-leading tools revealed a significant gap: existing platforms tend to offer either overly complex trading charts or static contract checks, failing to provide the comprehensive, easy-to-understand risk analysis that retail investors need.
2. **Solving Data Interoperability:** To address the challenge of handling data from multiple blockchains, a “Chain-Adaptive” architecture was successfully implemented. A key achievement was the development of a normalization layer that abstracts technical differences between networks. This allowed the system to process data from high-speed chains like Solana just as effectively as legacy layers like Ethereum, solving the core problem of data interoperability.
3. **Project Implementation:** The project that is live on tokenomiclab.app demonstrated that real-time financial analysis is feasible using a serverless-first strategy built on Next.js and Firebase. More importantly, integrating the Groq AI SDK (Llama 3.3) transformed the platform from a standard calculator into an intelligent advisory tool, capable of explaining complex risk scores in plain language.
4. **Validating the Risk Algorithm:** The custom 10-factor risk scoring model proved its reliability during rigorous testing. In controlled simulations against various token types, the algorithm achieved an 80% accuracy rate, successfully flagging malicious patterns such as liquidity manipulation and high holder concentration without human intervention.
5. **User-Centric Effectiveness:** Finally, beta testing with live users confirmed the platform’s practical value. Testers consistently found the visual risk gauge and AI summaries more effective than manual checks, validating that combining algorithmic scoring with automated explanations is a viable solution for preventing fraud in the decentralized finance sector.

REFERENCES

- Certik. (2024, December 6). Retrieved from <https://www.certik.com/resources/blog/skyknight-memescan-securing-solana-memecoins>
- Chainalysis Team. (2025, January 15). *2025 Crypto Crime Trends*. Retrieved from chainalysis: <https://www.chainalysis.com/blog/2025-crypto-crime-report-introduction/>
- Dextools. (n.d.). Retrieved from Dextools: <https://info.dextools.io/>
- Federal bureau of investigation. (2024). *Federal Bureau of Investigation. 2024 Internet Crime Report*. Retrieved from IC3: https://www.ic3.gov/AnnualReport/Reports/2024_IC3Report.pdf#page=3&zoom=100,92,81
- Finshots. (2025, 03 4). *Finshots*. Retrieved from Finshots: <https://finshots.in/archive/memecoin-fraud-just-became-easier-us-sec-securities-and-exchange-commission/>
- Firebase. (2025). *Firebase*. Retrieved from Firebase: <https://firebase.google.com/pricing>
- GDPR. (2016). Retrieved from gdpr-info: <https://gdpr-info.eu/>
- Ghattas, S. (2025, October). *Bitbond*. Retrieved from Bitbond: <https://www.bitbond.com/resources/dextools-the-ultimate-guide/>
- Google. (2025, December 7). Retrieved from <https://gemini.google.com/share/7103abc0262e>
- Goplus. (2025). *Goplus*. Retrieved from Goplus: <https://docs.gopluslabs.io/reference/api-overview>
- Gopluslabs. (2025). *Gopluslabs*. Retrieved from Gopluslabs: <https://gopluslabs.io/security-api>
- groq. (2025). Retrieved from <https://groq.com/pricing>
- Helius. (2025). *Helius*. Retrieved from Helius: <https://www.helius.dev/pricing>
- Josh, N. J. (2025, December 22). *Github*. Retrieved from Github: <https://github.com/lvingsarcophagus/Tokenomic-lab>
- Josh, N. J. (2025, December 22). *Perplexity*. Retrieved from Perplexity: https://www.perplexity.ai/search/table-of-risk-factors-and-calc-Dm68lAKqQ4qAbpivXEW_Fw#24
- Josh, N. J. (2025, December 22). *Tokenomics lab*. Retrieved from Tokenomics lab: <https://tokenomiclab.app/>
- Merkle Science reports \$500M lost to memecoin scams in 2024. (2025, January 11). Retrieved from Investpedia: <https://www.investing.com/news/cryptocurrency-news/merkle-science-reports-500m-lost-to-memecoin-scams-in-2024-93CH-3861901>
- Mobula. (2025). *docs.mobula.io*. Retrieved from <https://docs.mobula.io/pricing>

Nansen. (2025, November 10). *Nansen*. Retrieved from Nansen: <https://www.nansen.ai/post/nansen-the-leading-blockchain-intelligence-tool-for-onchain-data-in-2025>

Netlify. (2025). *Netlify*. Retrieved from Netlify: <https://www.netlify.com/pricing/>

Rob Behnke. (2023, July). *Explained: The BALD Token Rug Pull*. Retrieved from Halborn: <https://www.halborn.com/blog/post/explained-the-bald-token-rug-pull-july-2023>

Saher. (2024, November 20). *Bitbond*. Retrieved from <https://www.bitbond.com/resources/token-sniffer-comprehensive-guide-review/>

Stan Kaminsky. (2025, February 17). *Kaspersky daily*. Retrieved from Meme coins and NFTs: How not to lose your shirt: <https://www.kaspersky.com/blog/trump-memecoins-cybersecurity-advice/53025/>

Token Sniffer. (2025). *Token Sniffer*. Retrieved from Token Sniffer: <https://tokensniffer.readme.io/reference/supported-networks>

Vermaak, W. (2022). *coinmarketcap.com*. Retrieved from coinmarketcap.com: <https://coinmarketcap.com/academy/article/dextools-guide-how-to-use-its-decentralized-market-data-for-trading-and-investing>

wikipedia. (2025). *Wikipedia*. Retrieved from wikipedia: [https://en.wikipedia.org/wiki/\\$Trump](https://en.wikipedia.org/wiki/$Trump)

SUMMARY

The aim of this thesis is to design and implement "Tokenomics Lab," a holistic web-based information system dedicated to automating the risk analysis of cryptocurrency tokens across multiple blockchain networks. The research addresses the problem of fragmentation in the DeFi sector, where retail investors lack unified tools to detect scams like "rug pulls" and "honeypots."

To achieve this aim, the following objectives were set and accomplished: investigating common tokenomic risk factors, designing a custom weighted 10-factor risk scoring algorithm, developing the application using a serverless architecture, and validating the system's accuracy.

The research methodology followed an Agile/Scrum development framework. The system was built using a modern technology stack, including Next.js 16 for the frontend, Firebase for the backend database and authentication, and the Groq AI SDK (Llama 3.3) for natural language processing. A "Chain-Adaptive" architecture was implemented to normalize data from disparate sources (Ethereum, Solana, etc.). The system's performance was evaluated through algorithmic simulations against a dataset of known malicious and safe tokens, as well as qualitative beta testing with users.

The main result is a fully functional platform that aggregates real-time market and security data to generate a comprehensive risk profile. The system successfully calculates a risk score on a 0–100 scale and provides AI-generated narrative explanations. Testing demonstrated that the risk algorithm achieved an 80% accuracy rate in identifying scam tokens, validating the effectiveness of combining algorithmic scoring with Large Language Models.

The "Tokenomics Lab" platform successfully bridges the information gap in the DeFi market. It proves that complex on-chain data can be processed and simplified for retail investors without sacrificing analytical depth, providing a viable solution for automated fraud detection.

Scope of the thesis: The thesis consists of [104] pages, [16] tables, and [30] figures.

Keywords: Cryptocurrency, Tokenomics, Fraud Detection, DeFi, Artificial Intelligence, Risk Analysis.

ANNEXE

ANNEXE 1: FIRESTORE RULES

ANNEXE 2: MEME DETECTION PROMPT

ANNEXE 3: MEME DETECTION PROMPT

FIRESTORE RULES

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper functions
    function isAuthenticated() {
      return request.auth != null;
    }

    function isOwner(userId) {
      return isAuthenticated() && request.auth.uid == userId;
    }

    function isPremium(userId) {
      return isAuthenticated() && isOwner(userId) &&
get(/databases/{database}/documents/users/{userId}).data.plan ==
'PREMIUM';
    }

    function isPayPerUse(userId) {
      return isAuthenticated() && isOwner(userId) &&
get(/databases/{database}/documents/users/{userId}).data.plan ==
'PAY_PER_USE';
    }

    function isPaidUser(userId) {
      return isPremium(userId) || isPayPerUse(userId);
    }

    function hasCredits(userId) {
      return isAuthenticated() && isOwner(userId) &&
get(/databases/{database}/documents/users/{userId}).data.credits
> 0;
    }

    function isAdmin() {
      return isAuthenticated() &&
get(/databases/{database}/documents/users/{request.auth.uid}).da
ta.role == 'admin';
    }

    // Users collection

```



```

match /users/{userId} {
  allow read: if isOwner(userId) || isAdmin();
  allow create: if isOwner(userId);
  allow update: if isOwner(userId) || isAdmin();
  allow delete: if isAdmin(); // Only admins can delete
users
}

// Activity logs collection - NEW: Admin monitoring
match /activity_logs/{logId} {
  // Only admins can read activity logs
  allow read: if isAdmin();

  // Any authenticated user can create logs (for their own
activities)
  allow create: if isAuthenticated();

  // No updates or deletes allowed (audit trail integrity)
  allow update, delete: if false;
}

// Watchlist subcollection
match /watchlist/{userId}/tokens/{tokenAddress} {
  allow read: if isOwner(userId) || isAdmin();
  allow write: if isOwner(userId);
  allow delete: if isOwner(userId);
}

// Alerts subcollection (Premium gets smart alerts, Pay-
per-use gets basic alerts)
match /alerts/{userId}/notifications/{alertId} {
  allow read: if isOwner(userId) || isAdmin();
  allow create: if isPaidUser(userId); // Both Premium and
Pay-per-use can create alerts
  allow update: if isOwner(userId); // Can mark as
read/dismissed
  allow delete: if isOwner(userId);
}

// Analysis history subcollection
match /analysis_history/{userId}/scans/{scanId} {
  allow read: if isOwner(userId) || isAdmin();
  allow create: if isOwner(userId);
  allow update: if false; // Immutable
  allow delete: if isOwner(userId);
}

// Portfolio collection (Premium and Pay-per-use)
match /portfolio/{userId} {
  allow read: if isOwner(userId) || isAdmin();
  allow write: if isPaidUser(userId); // Both Premium and
Pay-per-use can use portfolio

```

```

        allow delete: if isOwner(userId);
    }

    // Notifications collection - NEW: For notification bell
    match /notifications/{notificationId} {
        // Users can read their own notifications
        allow read: if isAuthenticated() && resource.data.userId
== request.auth.uid;

        // System can create notifications
        allow create: if isAuthenticated();

        // Users can update/delete their own notifications
        allow update, delete: if isAuthenticated() &&
resource.data.userId == request.auth.uid;
    }

    // Admin notification preferences
    match /admin_notification_preferences/{userId} {
        allow read, write: if isOwner(userId) && isAdmin();
    }

    // Settings collection
    match /settings/{userId} {
        allow read: if isOwner(userId) || isAdmin();
        allow write: if isOwner(userId);
        allow delete: if false; // Never delete settings
    }

    // System settings (admin only)
    match /system/{document=**} {
        allow read: if isAuthenticated();
        allow write: if isAdmin();
    }

    // Token cache (read-only for users, write for
system/admin)
    match /token_cache/{tokenAddress} {
        allow read: if isAuthenticated();
        allow write: if isAdmin();
    }

    // Token cache collection (alternative naming)
    match /tokenCache/{tokenAddress} {
        allow read: if isAuthenticated();
        allow write: if isAdmin();
    }

    // Admin settings
    match /admin_settings/{settingId} {
        allow read: if isAdmin();
        allow write: if isAdmin();
    }

```

```

    }

    // Credit transactions - For PAY_PER_USE tier
    match /credit_transactions/{transactionId} {
        // Users can read their own transactions
        allow read: if isAuthenticated() && resource.data.userId
== request.auth.uid;

        // Admins can read all transactions
        allow read: if isAdmin();

        // System can create transactions (via API)
        allow create: if isAuthenticated();

        // No updates or deletes (audit trail)
        allow update, delete: if false;
    }

    // Payment history - For tracking x402 payments
    match /payment_history/{userId}/payments/{paymentId} {
        allow read: if isOwner(userId) || isAdmin();
        allow create: if isOwner(userId); // User can record
their payments
        allow update: if false; // Immutable payment records
        allow delete: if false; // Never delete payment history
    }

    // Feature usage tracking - For pay-per-use billing
    match /feature_usage/{userId}/usage/{usageId} {
        allow read: if isOwner(userId) || isAdmin();
        allow create: if isOwner(userId); // Track feature usage
        allow update: if false; // Immutable usage records
        allow delete: if false; // Never delete usage history
    }

    // x402 payment sessions - For tracking payment flows
    match /x402_sessions/{sessionId} {
        // Users can read their own payment sessions
        allow read: if isAuthenticated() && resource.data.userId
== request.auth.uid;

        // Admins can read all sessions
        allow read: if isAdmin();

        // System can create and update sessions
        allow create: if isAuthenticated();
        allow      update:      if      isAuthenticated()      &&
resource.data.userId == request.auth.uid;

        // No deletes (audit trail)
        allow delete: if false;
    }

```

```
// Legacy collections (backward compatibility)
match /tokens/{tokenId} {
  allow read: if isAuthenticated();
  allow write: if isAuthenticated();
}

match /watchlists/{userId} {
  allow read, write: if isOwner(userId);
}

// Deny all other access
match /{document=**} {
  allow read, write: if false;
}
}
```

MEME DETECTION PROMPT

You are a cryptocurrency analyst. Respond ONLY with valid JSON. No markdown, no code blocks, just raw JSON.

Analyze this cryptocurrency token and classify it. Return ONLY valid JSON.

Token Name: \${tokenName}

Symbol: \${tokenSymbol}

\${metadata?.description ? `Description: \${metadata.description}` : ""}

\${metadata?.website ? `Website: \${metadata.website}` : ""}

\${metadata?.twitter ? `Twitter: \${metadata.twitter}` : ""}

Classify as one of: MEME_TOKEN, UTILITY_TOKEN, GOVERNANCE_TOKEN, or UNKNOWN

Meme tokens typically:

- Have dog, cat, or animal themes
- Use internet memes or viral references
- Have community-focused names
- Lack clear utility beyond speculation

Return JSON format:

```
{
  "classification": "MEME_TOKEN | UTILITY_TOKEN | GOVERNANCE_TOKEN | UNKNOWN",
  "confidence": 0-100,
  "reasoning": "brief explanation"
}
```

PLAYWRIGHY UI TESTING SCRIPT

```

// @ts-check
const { defineConfig, devices } = require('@playwright/test');

/**
 * Playwright Configuration for Tokenomics Lab E2E Tests
 *
 * @see https://playwright.dev/docs/test-configuration
 */
module.exports = defineConfig({
  testDir: './tests',

  /* Run tests in files in parallel */
  fullyParallel: true,

  /* Fail the build on CI if you accidentally left test.only in the source code. */
  forbidOnly: !!process.env.CI,

  /* Retry on CI only */
  retries: process.env.CI ? 2 : 0,

  /* Opt out of parallel tests on CI. */
  workers: process.env.CI ? 1 : undefined,

  /* Reporter to use. See https://playwright.dev/docs/test-reporters */
  reporter: [
    ['html', { outputFolder: 'test-results/html-report' }],
    ['json', { outputFile: 'test-results/results.json' }],
    ['list']
  ],

  /* Shared settings for all the projects below. See https://playwright.dev/docs/api/class-testoptions. */
  use: {
    /* Base URL to use in actions like `await page.goto('/')`. */
    baseURL: process.env.BASE_URL || 'http://localhost:3000',

    /* Collect trace when retrying the failed test. See https://playwright.dev/docs/trace-viewer */
    trace: 'on-first-retry',

    /* Take screenshot on failure */
    screenshot: 'only-on-failure',

    /* Record video on failure */
    video: 'retain-on-failure',

    /* Global timeout for each action */
    actionTimeout: 15000,
  },
});

```

```

    /* Global timeout for navigation */
    navigationTimeout: 30000,
  },

  /* Configure projects for major browsers */
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },

    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },

    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },

    /* Test against mobile viewports. */
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 5'] },
    },
    {
      name: 'Mobile Safari',
      use: { ...devices['iPhone 12'] },
    },

    /* Test against branded browsers. */
    {
      name: 'Microsoft Edge',
      use: { ...devices['Desktop Edge'], channel: 'msedge' },
    },
    {
      name: 'Google Chrome',
      use: { ...devices['Desktop Chrome'], channel: 'chrome' },
    },
  ],

  /* Run your local dev server before starting the tests */
  webServer: {
    command: 'pnpm dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
    timeout: 120000,
  },

```

```
/* Global test timeout */
timeout: 60000,

/* Expect timeout */
expect: {
  timeout: 10000,
},

/* Output directory for test results */
outputDir: 'test-results/',
});
```